



République Tunisienne
Ministère de l'Enseignement Supérieur et de la
Recherche Scientifique
Université de Tunis El Manar
Faculté des Sciences de Tunis



MEMOIRE DE MASTER

Présenté en vue de l'obtention du
Diplôme National de Mastère Professionnel
En Système de Télécommunications et Réseaux

Réalisé par :

Yasmine Mouakhar

**Conception, développement et déploiement d'une
plateforme SaaS multi-tenant pour la gestion de
marketplaces bancaires avec mise en place d'un pipeline
CI/CD**

Soutenu le 26/06/2024

Devant le Jury :

Mr. Med Anouar Ben Messaoud	Maître de Conférences	Président
Mme. Rachida Bedira	Maître assistante	Examinatrice
Mr. Dhafer Mezghani	Maître de Conférences	Encadrant

Réalisé à :

BRI-Technology



Année Universitaire 2025 - 2026

Encadrant Académique

J'autorise l'étudiante à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant Académique : **Tarek CHAABANI**

Signature

Encadrant Professionnel

J'autorise l'étudiante à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant Professionnel : **Naja AGUILI**

Signature et cachet

Remerciements

Le travail exposé dans ce mémoire a été entrepris dans le cadre de la préparation du Diplôme National de Master Professionnel en Systèmes de Télécommunications et Réseaux à la Faculté des Sciences de Tunis.

Au thème de ce projet, je tiens à exprimer ma sincère gratitude envers toutes les personnes qui m'ont soutenue tout au long de mon parcours.

*A cette occasion, j'exprime ma profonde gratitude et mes sincères remerciements à Monsieur **Tarek Chabbani**, mon encadrant professionnel, pour avoir accepté de m'accompagner durant ce projet. Je le remercie pour son suivi, ses orientations pertinentes ainsi que pour les précieux conseils qu'il m'a prodigués tout au long de mon stage.*

*Je tiens également à remercier chaleureusement Madame **Naja Aguli**, mon encadrante professionnelle, pour son accompagnement, son écoute et son soutien. Ses remarques constructives, ses recommandations et les compétences qu'elle a su me transmettre ont largement contribué à l'amélioration de ce travail.*

*Mes remerciements s'adressent également à Monsieur **Riadh Bouhouchi**, administrateur de l'entreprise, pour son accueil, ses précieux conseils, et les moyens mis à ma disposition afin de mener ce projet dans les meilleures conditions.*

*Je remercie aussi l'ensemble de l'équipe de **BRI Technology** pour son soutien quotidien, sa disponibilité et la qualité de son accueil. L'esprit de collaboration de ses membres a facilité mon intégration au sein de l'entreprise et a rendu cette expérience particulièrement enrichissante, aussi bien sur le plan professionnel que personnel.*

Un remerciement particulier s'adresse aux membres du jury pour avoir accepté d'évaluer ce travail et de me faire bénéficier de leurs remarques, de leurs recommandations et de leurs conseils constructifs.

.

Dédicace

C'est avec une profonde émotion et une grande reconnaissance que je dédie ce travail à toutes les personnes qui ont contribué, de près ou de loin, à son accomplissement

A mes chers parents

Kamel & Fatma

Pour leur amour, leurs sacrifices, leur soutien permanent et leurs encouragements tout au long de mon parcours. Vous avez toujours été ma source de force et de motivation. J'espère rester toujours à la hauteur de votre confiance et ne jamais vous décevoir.

Que Dieu vous procure bonne santé et longue vie.

A mes chers frères

Hamdi & Yassine

Pour leur soutien moral, leurs encouragements et leurs précieux conseils, qui m'ont accompagnée durant toutes les étapes de ce travail.

A toute personne qui a marqué ma vie par un mot, un conseil ou un geste qui m'a guidée vers le bon chemin.

Merci pour tout l'amour et la confiance et pour votre énorme support pendant la réalisation de mon projet.

Table des matières

Table of Contents

<i>Liste des figures</i>	IX
<i>Liste des tableaux</i>	X
<i>Liste des acronymes</i>	XI
Introduction générale	1
Chapitre 1 : Cadre général du projet	3
1. Introduction	3
2. Présentation de l'organisme d'accueil	3
2.1. BRI Technology	3
2.2. Présentation des services offerts par BRI Technology	3
2.3. Les solutions proposées par BRI Technology	4
3. Présentation du projet	5
3.1. Contexte du projet	5
3.2. Étude de l'existant	5
3.3. La problématique	6
3.4. La solution proposée	7
4. Méthodologie de Gestion de Projet : SCRUM	7
4.1. Le cycle de vie de Scrum	7
4.2. L'équipe de Scrum	8
5. Planification du projet	8
5.1. Release 1 : Mise en œuvre de la plateforme SaaS et gestion du cycle de vie des marketplaces bancaires	9
Sprint	9
Objectif	9
Éléments clés du backlog	9
S1	9
Socle SaaS multi-tenant.....	9
Authentification, gestion des sessions, autorisation par rôles et application du contexte multi-tenant afin d'isoler les données de chaque banque.....	9
S2	9
Demande de création de marketplace.....	9
Informations de la banque, futur Administrateur Banque, vérification de l'e-mail, slug, identité visuelle, stores, modules, récapitulatif et soumission.....	9
S3	9

Back-office SaaS	9
Tableau de bord, Notifications, stores et modules, banques, utilisateurs et rôles, concessionnaires, marketplaces, contenus, abonnements, audit, logs et paramètres.....	9
S4	9
Traitement, paiement et activation	9
Consultation des demandes, approbation ou rejet avec motif, e-mails, lien Stripe, contrôle du paiement, activation de la banque, de la marketplace, de l'abonnement et du compte Administrateur Banque.....	9
S5	9
Back-office Banque et marketplace publique.....	9
Personnalisation et contenus, configuration des stores et modules, renouvellement ou ajout de services, marketplace public.....	9
5.2. Release 2 : Mise en œuvre des parcours des concessionnaires et des clients, et intégration des assistants conversationnels	9
Sprint	10
Objectif	10
Éléments clés du backlog	10
S6	10
Inscription et validation du concessionnaire	10
Demande d'inscription, dépôt des pièces justificatives, contrôle SaaS, création du compte et e-mail des identifiants.....	10
S7	10
Partenariats et contrats.....	10
Demandes initiées par la banque ou le concessionnaire, validation, cycle de vie du contrat et gestion des conditions de collaboration.....	10
S8	10
Produits, publications et suivi des dossiers	10
Catalogue centralisé, stock, publications par marketplace et consultation des demandes de financement associées aux produits.....	10
S9	10
Parcours client et demande de financement	10
Inscription et vérification du compte, consultation, comparaison, simulation, soumission d'une demande de financement et traitement du dossier.....	10
S10	10
Assistants conversationnels et validation	10
Assistant IA du SaaS, chatbot public à règles métier, contrôle des accès.	10
6. Conclusion	10
Chapitre 2 : Analyse et spécification des besoins	11

1. Introduction	11
2. Identification des Acteurs	11
3. Besoins fonctionnels	12
3.1. En tant que Administrateur SaaS	12
3.2. En tant que Administrateur Banque	13
3.3. En tant que Concessionnaire	13
3.4. En tant que Client.....	14
3.5. En tant que Internaute	14
4. Besoins non fonctionnels	15
5. Diagramme de cas d'utilisation global	15
6. Spécification détaillée des cas d'utilisation et scénarios.....	17
6.1. Cas d'utilisation détaillée : S'authentifier.....	17
6.2. Cas d'utilisation détaillée : Déposer une demande de marketplace.....	18
6.3. Cas d'utilisation détaillée : Traiter une demande et activer les services.....	20
6.4. Cas d'utilisation détaillée : Inscrire et valider un concessionnaire.....	21
6.5. Cas d'utilisation détaillée : Gérer et publier un produit concessionnaire.....	23
6.6. Cas d'utilisation détaillée : Déposer et traiter une demande de financement.....	25
7. Conclusion.....	27
Chapitre 3 : Architecture et Conception	28
1. Introduction	28
2. Architecture SaaS et de la multi-tenant	28
2.1. Présentation du SaaS.....	28
2.2. Principe de multi-tenant	28
2.3. Principales stratégies multi-tenant	29
2.4. Choix d'architecture	30
3. Architecture globale	31
3.1. Architecture logique.....	31
3.2. Architecture physique	32
4. Environnement de développement et technologies utilisées	33
4.1. Environnement matériel	33
4.2. Environnement logiciel	34
4.3. Technologies utilisées	35
5. Conclusion.....	37
Chapitre 4 : Mise en œuvre de la plateforme SaaS et gestion du cycle de vie des marketplaces bancaires	38
1. Introduction	38

2. Authentification, autorisation et contexte multi-tenant	38
3. Demande de création d'une marketplace bancaire	39
3.1. Renseignement des informations de la banque	40
3.2. Informations du futur Administrateur Banque et vérification de l'adresse e-mail	41
3.3. Configuration initiale de la marketplace, sélection des stores et modules	41
3.4. Récapitulatif et soumission de la demande.....	42
4. Back-office SaaS : administration et supervision de la plateforme	42
4.1. Tableau de bord.....	42
4.2. Gestion des stores et des modules.....	43
4.3. Gestion des demandes de marketplace	43
4.3.1. Traitement d'une demande de marketplace	43
4.3.2. Approbation et déclenchement du paiement.....	44
4.3.3. Paiement sécurisé par Stripe et activation de la marketplace.....	44
4.4. Gestion des banques	45
4.5. Gestions des utilisateurs et des rôles	45
4.6. Gestion des concessionnaires	45
4.7. Gestion des marketplaces	45
4.8. Gestion du contenu des marketplaces.....	45
4.9. Gestion des abonnements.....	46
4.10. Audit et logs	46
5. Back-office Banque : administration de la marketplace.....	46
5.1. Personnalisation de la marketplace	47
5.2. Gestion des stores, modules et paramètres.....	47
5.3. Renouvellement et l'ajout des services	48
6. Marketplace publique	48
7. Conclusion.....	49
Chapitre 5 : Mise en œuvre des parcours des concessionnaires et des clients, et intégration des assistants conversationnels	51
1. Introduction	51
2. Parcours concessionnaires	51
2.1. Inscription et validation du concessionnaire.....	51
2.2. Partenariats et contrats.....	52
2.3. Produits et publication multi-banque.....	53
2.4. Suivi des demandes de financement.....	53
3. Parcours client et demande de financement	54
3.1. Consultation, comparaison et simulation	54

3.2. Inscription du client et activation du compte.....	54
3.3. Constitution et soumission du dossier.....	55
3.4. Traitement de la demande par la banque	55
4. Assistants conversationnels de la plateforme.....	56
4.1. Assistant IA pour le pilotage du SaaS.....	56
4.2. Fonctionnement et sécurisation de l'assistant IA	56
4.3. Chatbot public des marketplaces	57
5. Introduction	57
Chapitre 6 : DevOps, pipeline CI/CD et déploiement sur Microsoft Azure.....	59
1. Introduction	59
2. Démarche DevOps et principes de la CI/CD.....	59
2.1. Intégration continue	59
2.2. Livraison et déploiement continus	60
3. Conteneurisation avec Docker.....	60
3.1. Principe de la conteneurisation	60
3.2. Dockerfile, image et conteneur	60
3.3. Conteneurisation du frontend et du backend	61
3.4. Orchestration avec Docker Compose	61
4. Mise en place du pipeline CI/CD avec Jenkins	61
4.1. Déclenchement automatique avec GitHub Webhook.....	62
4.2. Construction du backend et du frontend	62
4.3. Exécution des tests backend	62
5. Analyse de la qualité avec SonarQube.....	62
6. Construction et publication des images Docker	63
7. Déploiement sur Microsoft Azure	63
7.1. Choix de la plateforme Cloud.....	63
7.2. Déploiement avec Azure Container Apps	64
7.3. Base de données PostgreSQL sur Azure	64
7.4. Automatisation du déploiement	64
8. Architecture complète de déploiement	65
9. Conclusion.....	65

Liste des figures

Figure 1: Logo de BRI Technology	3
Figure 2: Cycle de vie de la méthodologie agile SCRUM	8
Figure 3: Principales stratégies d'architecture multi-tenant.....	Error! Bookmark not defined.

Liste des tableaux

Liste des acronymes

SaaS : Software as a Service

MVC : Model View Controller

JDBC : Java Database Connectivity

API : Application Programming Interface

SMTP : Simple Mail Transfer Protocol

NLP : Natural Language Processing

LLM : Large Language Model

CI : Continuous Integration

CD : Continuous Deployment

Introduction générale

La transformation numérique occupe aujourd'hui une place importante dans le secteur bancaire. Les établissements financiers cherchent à diversifier leurs services et à proposer à leurs clients des expériences digitales plus accessibles, personnalisées et adaptées à leurs besoins. Dans ce contexte, les marketplaces bancaires constituent une nouvelle approche permettant de regrouper, au sein d'un même espace numérique, différents produits et services proposés en collaboration avec plusieurs partenaires.

Toutefois, la mise en place et la gestion de ces marketplaces présentent plusieurs défis. Le développement d'une application spécifique pour chaque banque entraîne la multiplication des versions, la duplication des opérations de configuration ainsi qu'une maintenance plus complexe. De plus, la gestion séparée des concessionnaires, des produits, des stocks et des demandes de financement limite la centralisation des informations et rend l'évolution globale de la solution plus difficile.

C'est dans ce contexte que s'inscrit notre projet de fin d'études, réalisé au sein de BRI Technology. Il consiste à concevoir, développer et déployer Matchia, une plateforme SaaS multi-tenant destinée à la création et à la gestion de plusieurs marketplaces bancaires. Cette solution vise à centraliser les principales fonctionnalités et ressources communes tout en permettant à chaque banque de disposer d'une marketplace personnalisée et d'un espace de gestion propre, avec une séparation des données entre les différents tenants.

Afin de présenter de manière structurée les différentes étapes de réalisation du projet, ce rapport est organisé en six chapitres.

Le premier chapitre, intitulé « **Cadre général du projet** », présente l'organisme d'accueil, le contexte du projet, l'étude de l'existant, la problématique identifiée, la solution proposée ainsi que la méthodologie de travail adoptée.

Le deuxième chapitre, intitulé « **Analyse et spécification des besoins** », présente les acteurs intervenant dans la plateforme ainsi que les besoins fonctionnels et non fonctionnels du système. Il expose également le diagramme de cas d'utilisation global et détaille les principaux cas d'utilisation à travers leurs scénarios.

Le troisième chapitre, intitulé « **Architecture et conception** », présente les concepts liés au SaaS et à l'architecture multi-tenant, les principales stratégies de gestion des tenants ainsi que le choix architectural retenu pour Matchia. Il décrit ensuite l'architecture globale de la plateforme, l'environnement de développement et les technologies utilisées dans sa réalisation.

Le quatrième chapitre, intitulé « **Mise en œuvre de la plateforme SaaS et gestion du cycle de vie des marketplaces bancaires** », présente la réalisation des principales fonctionnalités liées à la création, à la configuration, à l'activation et à l'administration des marketplaces bancaires.

Le cinquième chapitre, intitulé « **Mise en œuvre des parcours des concessionnaires et des clients, et intégration des assistants conversationnels** », présente les fonctionnalités destinées aux concessionnaires et aux clients, notamment la gestion des partenariats et des produits, le parcours de demande de financement ainsi que les assistants conversationnels intégrés à la plateforme.

Enfin, le sixième chapitre, intitulé « **Mise en place de la démarche DevOps, du pipeline CI/CD et déploiement de Matchia sur Microsoft Azure** », décrit la démarche DevOps adoptée, la conteneurisation de l'application, l'automatisation de la construction, des tests et de l'analyse de la qualité du code, ainsi que le processus de déploiement de la plateforme sur Microsoft Azure.

Chapitre 1 : Cadre général du projet

1. Introduction

Ce projet est situé dans son contexte global en commençant par une introduction de l'organisme d'accueil. Nous examinons ensuite la situation actuelle pour identifier la solution appropriée au problème mentionné, tout en soulignant les objectifs à atteindre. Par la suite, nous étudierons la méthodologie appliquée.

2. Présentation de l'organisme d'accueil

2.1. BRI Technology

BRI Technology est une entreprise tunisienne fondée en 2010, spécialisée dans l'intelligence artificielle et la transformation digitale. Elle accompagne principalement les banques, les compagnies d'assurance, les opérateurs télécoms et les institutions publiques dans la modernisation de leurs services numériques. Grâce à son expertise technologique, l'entreprise propose des solutions intelligentes, sécurisées et évolutives adaptées aux besoins de chaque client. Elle intervient en Tunisie ainsi que dans plusieurs pays, notamment en Afrique, en Europe et en Amérique du Nord [1].



Figure 1: Logo de BRI Technology

2.2. Présentation des services offerts par BRI Technology

BRI Technology propose un ensemble de services destinés à accompagner les entreprises dans leur transformation numérique. Parmi ses principaux services, on distingue :

- **Conseil** : BRI Technology accompagne les organisations dans leur transformation numérique à travers la réalisation d'études stratégiques et l'élaboration de feuilles de route détaillées. Cette approche permet d'orienter l'évolution des systèmes d'information et de transformer les processus traditionnels en écosystèmes numériques modernes, efficaces et performants.
- **Recherche et développement** : BRI Technology intervient dans la conception, le développement et la mise en œuvre de solutions numériques avancées, adaptées aux

besoins spécifiques de différents secteurs d'activité. Ses réalisations couvrent notamment le commerce en ligne, les marketplaces et les plateformes digitales sur mesure. Chaque solution est conçue de manière personnalisée afin de répondre efficacement aux exigences et aux objectifs de chaque client.

- **Audit** : BRI Technology met à disposition son expertise pour analyser et évaluer les systèmes d'information existants. Elle accompagne également les établissements publics et privés dans la définition de plans directeurs stratégiques et opérationnels, favorisant ainsi l'amélioration continue, la performance et l'alignement avec les objectifs à long terme.
- **Formation et certification** : BRI Technology propose des programmes de formation adaptés aux besoins de ses clients et de leurs équipes. Ces formations sont accompagnées, si nécessaire, de services de certification permettant de valider les compétences acquises, de renforcer le savoir-faire des collaborateurs et d'assurer la conformité aux standards exigés.

2.3. Les solutions proposées par BRI Technology

Les solutions proposées par BRI Technology couvrent plusieurs domaines technologiques. L'entreprise développe des solutions d'intelligence artificielle et d'analyse de données permettant d'exploiter les données afin de faciliter la prise de décision, la prédiction et l'automatisation intelligente. Elle accompagne également les organisations dans leur stratégie de transformation digitale à travers l'optimisation des processus, la modernisation des systèmes existants et l'adoption de nouvelles technologies.

Dans le domaine bancaire, BRI Technology propose des solutions de digital banking permettant l'intégration des systèmes bancaires, la mise en place de plateformes web et mobiles, les solutions de paiement et les portefeuilles numériques. Son produit phare, **Matchia**, est une marketplace de financement intelligente basée sur l'intelligence artificielle. Cette solution permet la simulation de produits financiers, l'évaluation d'éligibilité en temps réel, l'onboarding client automatisé et l'intégration avec différents partenaires.

L'entreprise propose également des solutions de cybersécurité, de développement logiciel sur mesure, d'applications web et mobiles, d'architectures API et microservices, ainsi que des solutions cloud natives. Ces solutions visent à offrir aux entreprises des plateformes modernes, performantes, sécurisées et capables d'évoluer selon leurs besoins.

3. Présentation du projet

3.1. Contexte du projet

Le secteur bancaire connaît une évolution continue portée par la digitalisation des services et par l'apparition de nouveaux usages numériques. Dans ce contexte, les banques cherchent à enrichir leur offre en proposant à leurs clients des services complémentaires accessibles à travers des plateformes digitales.

Les marketplaces bancaires s'inscrivent dans cette évolution en permettant de regrouper, au sein d'un même environnement, différents produits et services proposés en collaboration avec plusieurs partenaires. Leur mise en œuvre nécessite toutefois une organisation capable de gérer efficacement la diversité des acteurs, des offres et des configurations associées à chaque établissement bancaire.

Avec l'augmentation du nombre de banques et de partenaires, la gestion de ces environnements peut rapidement devenir complexe. Il devient alors essentiel d'adopter une approche permettant de simplifier leur administration, de faciliter leur évolution et d'améliorer la centralisation des différentes opérations.

3.2. Étude de l'existant

Avant la mise en place de la solution proposée, la gestion des marketplaces bancaires reposait principalement sur une approche spécifique à chaque établissement. Chaque banque disposait de son propre environnement, configuré et adapté selon ses besoins.

Cette organisation permettait de répondre aux besoins particuliers de chaque banque, mais elle reposait fortement sur des interventions manuelles et sur la multiplication d'environnements indépendants. Avec l'augmentation du nombre de banques et de partenaires, plusieurs limites sont apparues.

Les principales caractéristiques de cette solution existante peuvent être résumées comme suit :

- **Marketplace spécifique à chaque banque :** Dans l'approche existante, chaque banque disposait d'une version spécifique et personnalisée de la marketplace, adaptée manuellement par l'équipe de développement selon ses besoins.

Chaque nouvelle mise en place nécessitait donc une configuration dédiée ainsi que, dans certains cas, des modifications ou des corrections du code. Cette multiplication des versions rendait la maintenance et l'évolution plus complexes, puisque certaines modifications devaient être reproduites sur plusieurs environnements.

- **Demande de marketplace et onboarding traditionnel :** La création d'une marketplace nécessitait des échanges directs entre la banque et le fournisseur de la solution afin de définir les besoins, les services souhaités et les différentes configurations. Ce processus restait donc dépendant de plusieurs interventions humaines.
- **Paiement non intégré :** Le paiement n'était pas intégré au processus de gestion de la marketplace et était traité manuellement ou à travers un système externe. Il n'était donc pas directement lié à l'activation automatique des services, ce qui nécessitait une intervention humaine pour confirmer le règlement, activer les services concernés et assurer le suivi des abonnements.
- **Gestion séparée des concessionnaires :** La gestion des concessionnaires était répartie entre les différentes marketplaces. Lorsqu'un concessionnaire collaborait avec plusieurs banques, il devait gérer un compte et un espace distinct pour chaque marketplace. Les informations sur les produits et les stocks devaient alors être mises à jour dans plusieurs environnements, ce qui entraînait une dispersion des données et une charge de gestion supplémentaire.

3.3. La problématique

L'étude de l'existant met en évidence plusieurs limites majeures :

- **Multiplication des applications et complexité de maintenance :** chaque banque disposant d'une marketplace spécifique, les évolutions et corrections devaient être reproduites sur plusieurs environnements.
- **Processus de création peu automatisé :** la mise en place d'une nouvelle marketplace nécessitait plusieurs échanges et interventions entre la banque et le fournisseur de la solution.
- **Paiement et activation non intégrés :** le règlement, sa vérification, la mise à jour de l'abonnement et l'activation des services nécessitaient des opérations supplémentaires.

- **Gestion décentralisée des concessionnaires** : un même concessionnaire devait gérer plusieurs comptes, produits et stocks dans différentes marketplaces, entraînant une dispersion des données et une charge de gestion supplémentaire.

3.4. La solution proposée

Afin de répondre aux besoins identifiés, la solution proposée consiste à mettre en place **Matchia**, une plateforme **SaaS multi-tenant** permettant de centraliser la gestion de plusieurs marketplaces bancaires au sein d'un même système. Chaque banque dispose de son propre espace personnalisable, avec ses stores, modules, contenus et paramètres, tout en s'appuyant sur un socle applicatif commun. La plateforme intègre également le processus d'adhésion en regroupant les étapes de demande, de validation, de paiement et d'activation dans un même parcours. Par ailleurs, la gestion des concessionnaires est centralisée grâce à un compte unique leur permettant de collaborer avec plusieurs banques et de gérer leurs produits et partenariats depuis un seul espace. La solution offre ainsi une plateforme centralisée, configurable, automatisée et évolutive, adaptée à la gestion de plusieurs établissements bancaires et de leurs différents partenaires.

4. Méthodologie de Gestion de Projet : SCRUM

Dans le cadre de notre projet, nous avons choisi d'appliquer la méthodologie agile **Scrum** pour structurer le développement. Le travail a ainsi été réparti en plusieurs sprints, chacun regroupant un ensemble cohérent de fonctionnalités à concevoir, développer et valider. Cette organisation nous a permis d'assurer un avancement progressif du projet, de mieux gérer les priorités et d'intégrer les ajustements nécessaires au fur et à mesure de l'évolution des besoins.

4.1. Le cycle de vie de Scrum

Le cycle de vie Scrum débute par la définition des **User Stories** par le Product Owner, qui les ajoute et les priorise dans le **Product Backlog** selon leur importance et leur valeur métier. Lors de la **Sprint Planning**, l'équipe sélectionne les User Stories à réaliser pendant le sprint et les décompose en tâches constituant le **Sprint Backlog**. Le sprint démarre ensuite pour une durée déterminée, généralement comprise entre deux et quatre semaines. Durant cette période, l'équipe réalise les tâches planifiées et organise quotidiennement un **Daily Scrum** afin de suivre l'avancement et d'identifier les éventuels obstacles. À la fin du sprint, l'incrément développé

est présenté et évalué lors de la **Sprint Review**, puis l'équipe analyse son organisation et les améliorations possibles durant la **Sprint Retrospective**. Le cycle reprend ensuite avec la planification du sprint suivant, permettant ainsi une évolution progressive et continue du produit [2].

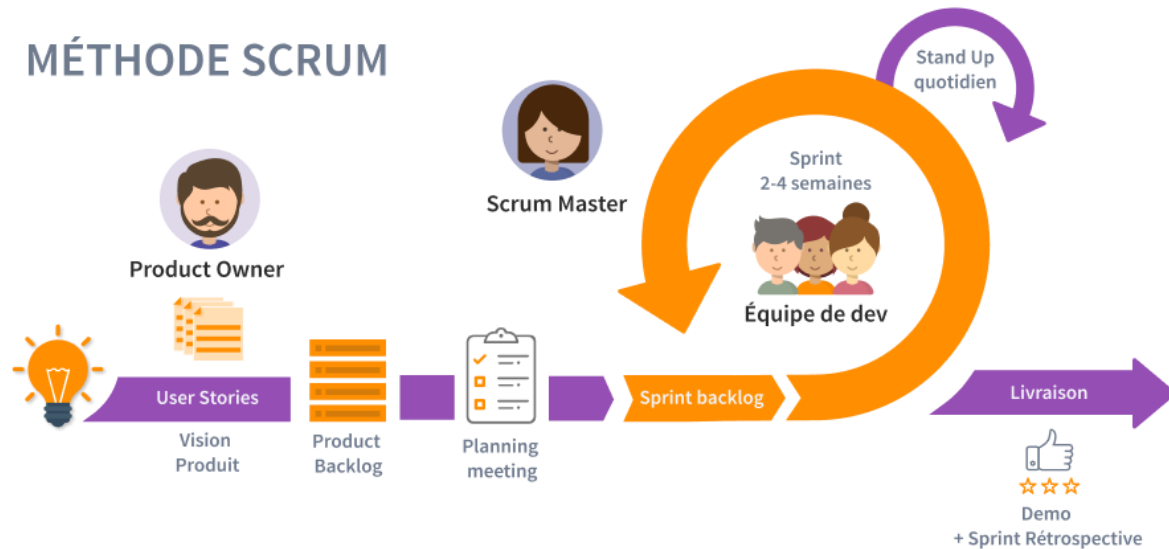


Figure 2: Cycle de vie de la méthodologie agile SCRUM

4.2. L'équipe de Scrum

Le projet repose sur les principaux rôles définis par la méthodologie SCRUM:

- **Product Owner** : identifie les besoins fonctionnels, définit les priorités du Product Backlog et valide les fonctionnalités réalisées.
- **Scrum Master** : facilite l'application de Scrum, accompagne l'équipe et veille au bon déroulement des différentes itérations.
- **Équipe de développement** : assure la conception, le développement, les tests et l'intégration des fonctionnalités de la plateforme.

5. Planification du projet

Afin d'organiser la réalisation de la solution de manière progressive et structurée, le projet a été réparti en deux releases, chacune regroupant plusieurs sprints. Cette planification permet de regrouper les fonctionnalités en ensembles cohérents, de suivre l'avancement du projet de manière itérative et d'assurer la réalisation progressive des différents blocs fonctionnels de la solution.

5.1. Release 1 : Mise en œuvre de la plateforme SaaS et gestion du cycle de vie des marketplaces bancaires

Cette release couvre la mise en place des principales fonctionnalités nécessaires au fonctionnement de la plateforme. Elle comprend notamment le socle multi-tenant, le processus de création de marketplace, l'administration SaaS, la gestion du paiement ainsi que la mise en œuvre des espaces bancaire et public.

Sprint	Objectif	Éléments clés du backlog
S1	Socle SaaS multi-tenant	Authentification, gestion des sessions, autorisation par rôles et application du contexte multi-tenant afin d'isoler les données de chaque banque.
S2	Demande de création de marketplace	Informations de la banque, futur Administrateur Banque, vérification de l'e-mail, slug, identité visuelle, stores, modules, récapitulatif et soumission.
S3	Back-office SaaS	Tableau de bord, Notifications, stores et modules, banques, utilisateurs et rôles, concessionnaires, marketplaces, contenus, abonnements, audit, logs et paramètres.
S4	Traitement, paiement et activation	Consultation des demandes, approbation ou rejet avec motif, e-mails, lien Stripe, contrôle du paiement, activation de la banque, de la marketplace, de l'abonnement et du compte Administrateur Banque.
S5	Back-office Banque et marketplace publique	Personnalisation et contenus, configuration des stores et modules, renouvellement ou ajout de services, marketplace public.

5.2. Release 2 : Mise en œuvre des parcours des concessionnaires et des clients, et intégration des assistants conversationnels

Cette release couvre les principales fonctionnalités liées aux concessionnaires et aux clients. Elle intègre la gestion des inscriptions, des partenariats et des produits. Elle prend également en charge le dépôt et le traitement des demandes de financement. Enfin, elle comprend l'intégration de l'assistant IA et du chatbot de la marketplace.

Sprint	Objectif	Éléments clés du backlog
S6	Inscription et validation du concessionnaire	Demande d'inscription, dépôt des pièces justificatives, contrôle SaaS, création du compte et e-mail des identifiants.
S7	Partenariats et contrats	Demandes initiées par la banque ou le concessionnaire, validation, cycle de vie du contrat et gestion des conditions de collaboration.
S8	Produits, publications et suivi des dossiers	Catalogue centralisé, stock, publications par marketplace et consultation des demandes de financement associées aux produits.
S9	Parcours client et demande de financement	Inscription et vérification du compte, consultation, comparaison, simulation, soumission d'une demande de financement et traitement du dossier.
S10	Assistants conversationnels et validation	Assistant IA du SaaS, chatbot public à règles métier, contrôle des accès.

6. Conclusion

En conclusion, ce chapitre a établi les bases du projet en présentant l'entreprise d'accueil, BRI-Technology. Il a également inclus une étude de l'existant, défini la problématique à résoudre et introduit la solution proposée ainsi que la méthodologie adoptée.

Chapitre 2 : Analyse et spécification des besoins

1. Introduction

Ce chapitre est consacré à l'analyse et à la spécification des besoins de la plateforme. Il présente les différents acteurs du système, leurs besoins fonctionnels et non fonctionnels, ainsi que les principaux choix liés à l'architecture SaaS multi-tenant.

2. Identification des Acteurs

L'identification des acteurs permet de déterminer les différents intervenants de la plateforme et de préciser leur rôle dans le système. Elle constitue une étape essentielle pour structurer les besoins fonctionnels et définir les interactions propres à chaque type d'utilisateur.

Tableau 1: Acteurs de la plateforme Matchia

Acteur	Description
Administrateur SaaS	Acteur principal chargé de l'administration globale de la plateforme. Il gère notamment les banques, les demandes d'adhésion, les demandes concessionnaires, les stores, les modules, les abonnements, les paiements, ainsi que la configuration générale du système.
Administrateur Banque	Responsable de la gestion de la marketplace de sa banque. Il administre les stores, les modules, les produits, les contenus, les clients, les concessionnaires partenaires et les demandes de financement.
Concessionnaire	Partenaire commercial pouvant collaborer avec plusieurs banques depuis un espace centralisé. Il gère ses produits ainsi que ses demandes, invitations et partenariats avec les banques.
Client	Utilisateur authentifié de la marketplace bancaire. Il peut gérer son profil, effectuer des simulations, déposer des

	demandes de financement, fournir les documents nécessaires et suivre l'état de ses demandes.
Internaute	Utilisateur non authentifié pouvant consulter les marketplaces, les stores et les produits, ainsi qu'utiliser certaines fonctionnalités publiques telles que la comparaison et la simulation.

3. Besoins fonctionnels

Les besoins fonctionnels traduisent les capacités que la plateforme doit offrir à chacun de ses acteurs. Ils permettent de structurer les fonctionnalités par profil utilisateur et de délimiter précisément les opérations accessibles selon les responsabilités et les droits associés à chaque rôle.

3.1. En tant que Administrateur SaaS

L'Administrateur SaaS dispose du niveau d'administration le plus élevé. Il assure la gestion globale de la plateforme et supervise les différents tenants.

Ses principales fonctionnalités sont les suivantes :

- S'authentifier et gérer son profil ;
- consulter le tableau de bord global de la plateforme ;
- gérer les stores et les modules;
- gérer les demandes d'adhésion des banques ;
- gérer les utilisateurs
- affecter un espace backoffice pour la demande approuvée
- gérer les banques enregistrées dans la plateforme ;
- gérer les marketplaces associées aux banques ainsi que leur contenu. ;
- gérer les abonnements ;
- consulter et traiter les demandes de création de comptes concessionnaires ;
- consulter les notifications ;
- utiliser l'assistant intelligent pour interroger certaines informations autorisées de la plateforme ;

- Consulter les logs et l'historique d'audit
- gérer les principales configurations de la plateforme ;

L'espace SaaS constitue ainsi le point central d'administration permettant de superviser l'ensemble de l'écosystème Matchia.

3.2. En tant que Administrateur Banque

L'Administrateur Banque assure la gestion de la marketplace appartenant à son établissement.

Il doit notamment pouvoir :

- s'authentifier et gérer son profil
- consulter le tableau de bord
- gérer les utilisateurs
- gérer la personnalisation de la marketplace
- gérer les contenus de la marketplace
- gérer les stores associés et activés pour sa marketplace
- gérer et configurer les modules disponibles dans chaque store
- gérer les concessionnaires partenaires
- gérer les demandes de partenariat envoyées et les invitations reçues
- gérer les contrats de partenariat
- Valider les produits proposés par les concessionnaires avant leur publication sur la marketplace.
- gérer les abonnements et suivre les services souscrits
- gérer les demandes de financement
- Consulter les notifications

L'administrateur Banque agit uniquement dans le périmètre de son établissement et ne doit pas accéder aux données appartenant aux autres banques.

3.3. En tant que Concessionnaire

Le Concessionnaire dispose d'un espace centralisé lui permettant de collaborer avec plusieurs banques. Il doit pouvoir :

- s'inscrire et accéder à son espace après validation
- s'authentifier et gérer son profil
- consulter son tableau de bord de son espace

- gérer ses produits et leurs stocks
- gérer ses publications auprès des banques partenaires
- gérer les demandes de partenariat envoyées et les invitations reçues
- gérer les partenariats et consulter les contrats associés avec les banques partenaires
- Consulter les notifications

L'un des besoins majeurs de cet espace est de permettre au concessionnaire de travailler avec plusieurs banques à partir d'un compte unique, sans multiplier les comptes et les espaces d'administration.

Cette centralisation facilite également la gestion des produits et réduit les opérations répétitives liées aux différents partenariats bancaires.

3.4. En tant que Client

Le Client est un utilisateur authentifié d'une marketplace bancaire. Son espace est principalement orienté vers les demandes de financement.

Le client doit pouvoir :

- S'inscrire, s'authentifier et gérer son profil
- Consulter les produits et leurs détails
- Comparer les produits disponibles
- Effectuer une simulation de financement
- Créer et soumettre une demande de financement
- Gérer les documents associés à la demande
- Consulter et suivre l'état de ses demandes
- Recevoir les notifications liées au traitement de ses demandes

3.5. En tant que Internaute

L'Internaute représente un utilisateur non authentifié consultant les espaces publics des marketplaces.

Il peut notamment :

- accéder aux marketplaces bancaires publiques;
- consulter les stores disponibles ;

- consulter les produits proposés ;
- comparer les produits disponibles;
- utiliser le simulateur de financement;

L'internaute peut consulter et simuler sans être connecté. En revanche, lorsqu'il souhaite déposer une demande de financement, le système doit l'inviter à s'inscrire ou à s'authentifier.

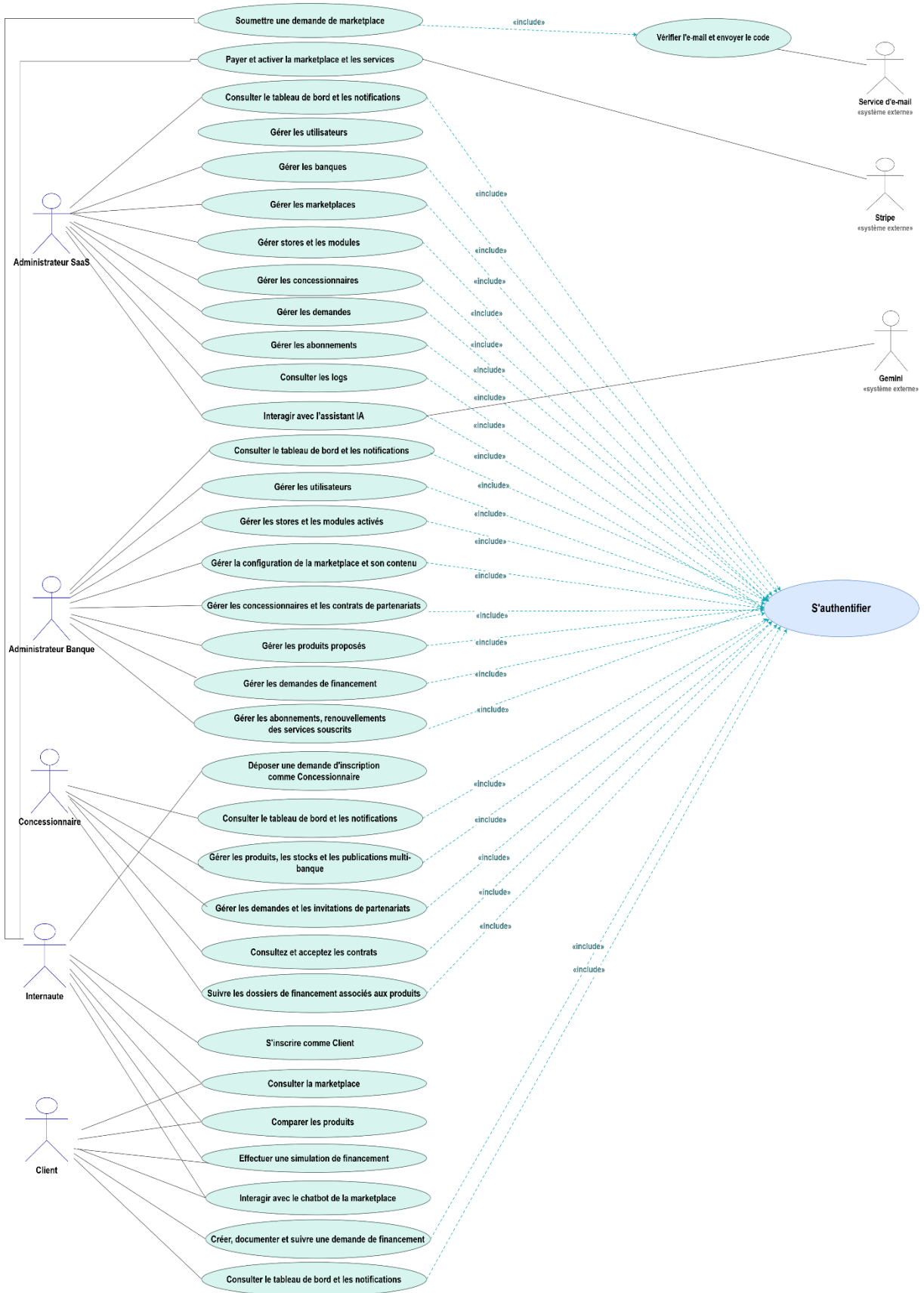
4. Besoins non fonctionnels

Les besoins non fonctionnels définissent les qualités attendues de la plateforme et les exigences nécessaires pour assurer un fonctionnement fiable, sécurisé, performant et adapté à une architecture SaaS multi-tenant. Parmi les principaux besoins non fonctionnels retenus pour la solution proposée, nous citons :

- **Sécurité** : La plateforme doit assurer une authentification sécurisée des utilisateurs et appliquer un contrôle d'accès basé sur les rôles. La séparation logique des données entre les différentes banques doit également être garantie afin d'éviter tout accès non autorisé aux informations d'un autre tenant.
- **Scalabilité** : La solution doit être capable d'intégrer progressivement de nouvelles banques, marketplaces, concessionnaires, d'utilisateurs et de données, sans nécessiter la création d'une nouvelle application pour chaque établissement.
- **Performance** : La solution doit assurer des temps de réponse satisfaisants et une navigation fluide.
- **Maintenabilité** : L'architecture de la plateforme doit faciliter les corrections, les évolutions et l'ajout de nouvelles fonctionnalités.
- **Disponibilité** : La plateforme doit garantir une disponibilité suffisante des services et assurer un fonctionnement stable des différents composants.

5. Diagramme de cas d'utilisation global

Dans le diagramme de cas d'utilisation global, nous regroupons l'ensemble des cas d'utilisation afin d'obtenir une vue d'ensemble du fonctionnement de notre solution et de mettre en évidence les différentes relations qui peuvent les relier.



6. Spécification détaillée des cas d'utilisation et scénarios

Les cas d'utilisation détaillés sont présentés à travers plusieurs diagrammes, chacun décrivant un processus métier spécifique et les interactions entre les acteurs concernés et le système.

6.1. Cas d'utilisation détaillée : S'authentifier

Ce cas d'utilisation décrit le processus d'authentification. La figure et le tableau associés présentent les acteurs concernés, les principales étapes du processus ainsi que les conditions d'accès aux différents espaces de la plateforme.

Accès sécurisé et contexte multi-tenant

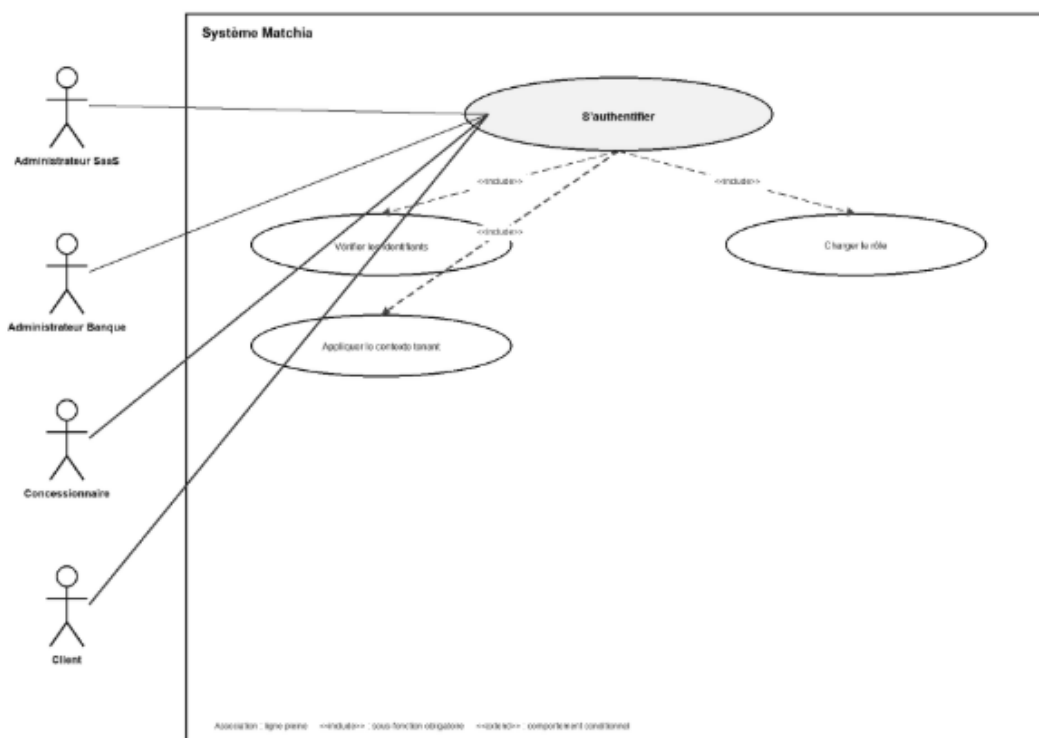


Figure 3.5 — Diagramme de cas d'utilisation détaillé de l'authentification

Le tableau suivant présente le scénario du cas d'utilisation UC-01 — S'authentifier.

Tableau 3.1 — Description du scénario UC-01

Élément	Description
Acteur principal	Utilisateur
Acteur secondaire	
Objectifs	Permettre à un utilisateur authentifié d'accéder à l'espace correspondant à son rôle et à son périmètre d'autorisation.

Préconditions	L'utilisateur dispose d'un compte actif et d'identifiants valides.
Postconditions	L'utilisateur dispose d'une session authentifiée avec les autorisations correspondant à son rôle et au tenant auquel il appartient.
Scénario nominal	1. L'utilisateur saisit son adresse e-mail et son mot de passe. 2. Le système vérifie les identifiants fournis. 3. Le système vérifie l'état du compte et récupère le rôle de l'utilisateur. 4. Pour les utilisateurs rattachés à une banque, le système détermine le contexte du tenant correspondant. 5. Une session authentifiée est créée. 6. L'utilisateur est redirigé vers l'espace correspondant à son rôle.
Alternatives et exceptions	Si les identifiants sont incorrects, le système refuse l'authentification et affiche un message d'erreur. Si le compte est inactif, l'accès est refusé. Si la session ou le jeton d'authentification a expiré, l'utilisateur doit renouveler sa session ou se reconnecter. Toute tentative d'accès à une ressource non autorisée ou appartenant à un autre tenant est refusée.

6.2. Cas d'utilisation détaillée : Déposer une demande de marketplace

Ce cas d'utilisation décrit le processus de soumission d'une demande de création de marketplace bancaire. La figure et le tableau associés présentent les acteurs concernés, les principales étapes du processus ainsi que les contrôles effectués avant l'enregistrement de la demande.

Demande de création d'une marketplace bancaire

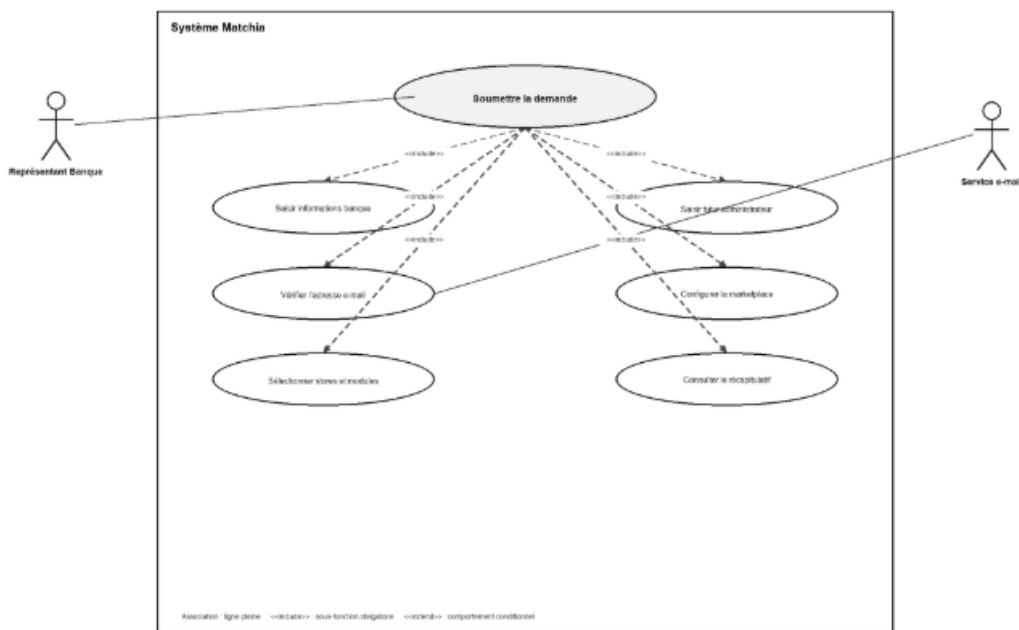


Figure 3.6 — Diagramme de cas d'utilisation détaillé de la demande de création d'une marketplace

Le tableau suivant présente le scénario du cas d'utilisation **UC-02 — Déposer une demande de marketplace**.

Tableau 3.2 — Description du scénario UC-02

Élément	Description
Acteur principal	Représentant de la banque
Acteur secondaire	Service e-mail
Objectifs	Permettre à une banque de transmettre une demande complète en vue de la création et de l'activation future de sa marketplace.
Préconditions	Le formulaire public de demande est accessible et l'adresse e-mail du futur Administrateur Banque n'est pas déjà associée à un compte existant.
Postconditions	La demande est enregistrée avec le statut « en attente » et devient accessible à l'Administrateur SaaS pour traitement. Une notification est générée pour informer l'Administrateur SaaS de la réception d'une nouvelle demande. Un e-mail de confirmation est également envoyé au représentant de la banque afin de l'informer que sa demande a été soumise avec succès.

Scénario nominal	1. L’Internaute renseigne les informations relatives à la banque et du futur Administrateur Banque. 3. Le système envoie un code de vérification à l’adresse e-mail renseignée. 4. L’internaute saisit et valide le code reçu. 5. Il renseigne configure le slug, l’identité visuelle, les stores et les modules souhaités. 6. Le système affiche un récapitulatif de la demande. 7. L’Internaute confirme et soumet la demande. 8. Le système enregistre la demande, crée la notification et envoie l’email de confirmation.
Alternatives et exceptions	Si une information obligatoire est manquante ou invalide, le système demande sa correction avant de poursuivre. Si le code de vérification est incorrect ou expiré, la validation de l’adresse e-mail échoue et un nouveau code peut être demandé. Si l’adresse e-mail est déjà utilisée, la demande ne peut pas être poursuivie avec cette adresse. Avant la soumission, le représentant peut modifier les stores, les modules ou les informations renseignées.

6.3. Cas d’utilisation détaillée : Traiter une demande et activer les services

Ce cas d’utilisation décrit le processus de traitement d’une demande de création de marketplace, La figure et le tableau associés présentent les acteurs concernés, les principales étapes du processus.

Figure 3.8 — Diagramme de cas d’utilisation détaillé du traitement, du paiement et de l’activation

Le tableau suivant présente le scénario du cas d’utilisation **UC-03 — Traiter une demande et activer les services**.

Tableau 3.4 — Description du scénario UC-03

Élément	Description
Acteur principal	Administrateur SaaS

Acteur secondaire	Stripe et service e-mail
Objectifs	Permettre à l'Administrateur SaaS de traiter une demande et d'activer les ressources associées uniquement après confirmation du paiement.
Préconditions	Une demande complète existe avec le statut « en attente ».
Postconditions	En cas de paiement confirmé, la demande est validée et la banque ainsi que les services associés sont activés. En cas de rejet, la demande est enregistrée comme rejetée avec le motif correspondant et le représentant de la banque en est informé.
Scénario nominal	1. L'Administrateur SaaS consulte et vérifie les informations de la demande. 2. Il approuve la demande. 3. Le système génère un lien de paiement via Stripe. 4. Un e-mail contenant le lien de paiement est envoyé au représentant de la banque. 5. Le représentant effectue le paiement. 6. Le système vérifie la confirmation du paiement auprès de Stripe. 7. Après confirmation, la banque, la marketplace, l'abonnement et le compte Administrateur Banque sont activés. 8. Un e-mail contenant les identifiants de connexion est envoyé à l'Administrateur Banque.
Alternatives et exceptions	Si la demande n'est pas conforme, l'Administrateur SaaS la rejette en précisant un motif et un e-mail de rejet est envoyé au représentant de la banque. Si le paiement échoue, est annulé ou n'est pas confirmé, les ressources associées restent inactives.

6.4. Cas d'utilisation détaillée : Inscrire et valider un concessionnaire

Ce cas d'utilisation décrit le processus d'inscription d'un concessionnaire, depuis le dépôt de sa demande jusqu'à sa validation par l'Administrateur SaaS. La figure et le tableau associés

présentent les acteurs concernés, les principales étapes du processus avant l'activation du compte.

Inscription et validation d'un concessionnaire

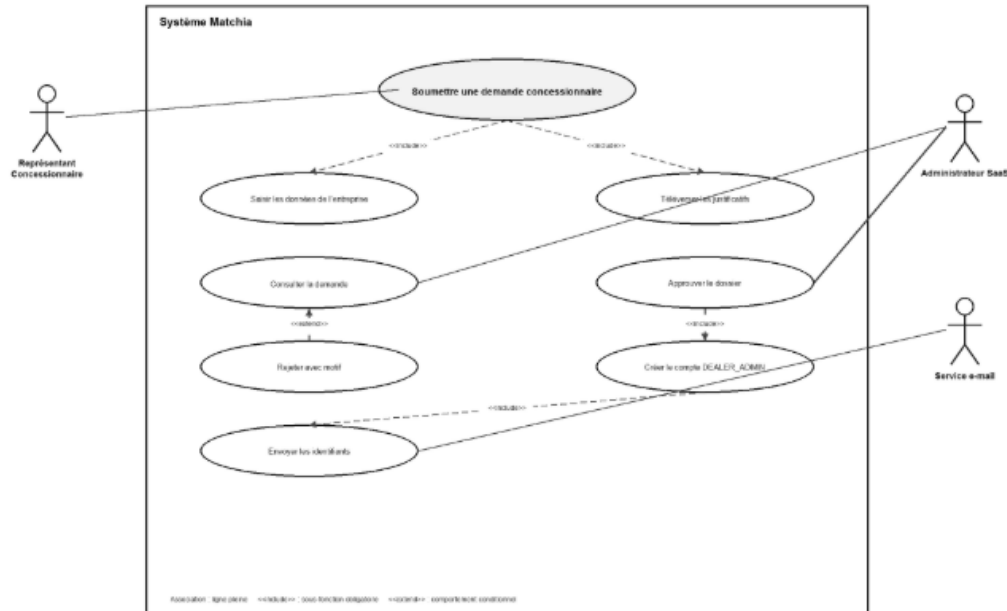


Figure 3.10 — Diagramme de cas d'utilisation détaillé de l'inscription et de la validation d'un concessionnaire

Le tableau suivant présente le scénario du cas d'utilisation **UC-06 — Inscrire et valider un concessionnaire**.

Tableau 3.6 — Description du scénario UC-04

Élément	Description
Acteur principal	Représentant du concessionnaire
Acteur secondaire	Administrateur SaaS et service e-mail
Objectifs	Obtenir un compte concessionnaire après la validation du dossier par l'Administrateur SaaS.
Préconditions	Le formulaire public d'inscription est accessible
Postconditions	En cas d'approbation, le concessionnaire dispose d'un compte actif lui permettant d'accéder à son espace. En cas de rejet, la demande est enregistrée comme rejetée avec le motif correspondant, et l'internaute en est informé.
Scénario nominal	1. L'Internaute renseigne les informations du concessionnaire.

	2. Il dépose les pièces justificatives demandées puis soumet la demande. 4. Le système enregistre la demande avec le statut « en attente ». 5. L'Administrateur SaaS consulte et vérifie le dossier. 6. Il approuve la demande. 7. Le système crée et active le compte Administrateur Concessionnaire. 8. Un e-mail contenant les identifiants de connexion est envoyé au l'internaute.
Alternatives et exceptions	Si des informations ou des documents obligatoires sont manquants ou invalides, la demande ne peut pas être soumise. Si le dossier n'est pas conforme, l'Administrateur SaaS rejette la demande en précisant un motif. Un e-mail informant du rejet et de son motif est alors envoyé au représentant du concessionnaire.

6.5. Cas d'utilisation détaillée : Gérer et publier un produit concessionnaire

Ce cas d'utilisation décrit le processus de gestion et de publication des produits par un concessionnaire, depuis la création du produit jusqu'à sa validation par une banque partenaire. La figure et le tableau associés présentent les acteurs concernés, les principales étapes du processus ainsi que les conditions nécessaires à la publication d'un produit dans une marketplace.

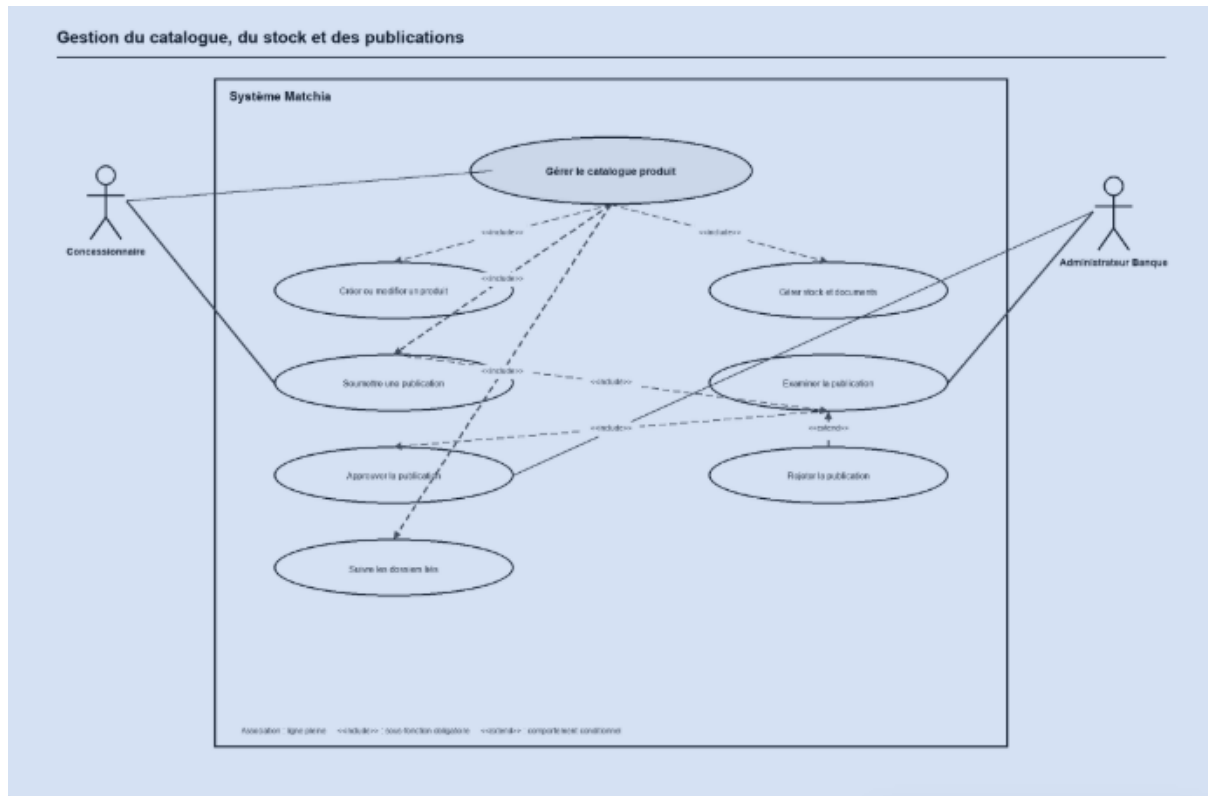


Figure 3.12 — Diagramme de cas d'utilisation détaillé de la gestion et de la publication des produits

Le tableau suivant présente le scénario du cas d'utilisation **UC-05 — Gérer et publier un produit concessionnaire**.

Tableau 3.8 — Description du scénario UC-05

Élément	Description
Acteur principal	Concessionnaire
Acteur secondaire	Administrateur Banque
Objectifs	Permettre au concessionnaire de gérer son catalogue de produits et de proposer leur publication dans les marketplaces des banques partenaires.
Préconditions	Le compte du concessionnaire est actif. Un partenariat et un contrat actifs existent avec la banque ciblée.
Postconditions	En cas d'approbation, le produit devient visible dans la marketplace de la banque concernée. En cas de rejet, le produit reste dans le catalogue du concessionnaire sans être publié dans

	cette marketplace. Le stock du produit reste géré de manière centralisée par le concessionnaire.
Scénario nominal	1. Le concessionnaire crée un nouveau produit ou modifie un produit existant. 2. Il renseigne les informations, les documents et le stock du produit. 3. Il sélectionne une banque partenaire et soumet le produit pour publication. 4. L'Administrateur Banque consulte la proposition de publication. 5. Il vérifie les informations du produit puis approuve la publication. 6. Le système publie le produit dans la marketplace de la banque concernée. 7. Le concessionnaire peut suivre ses publications et consulter les demandes de financement associées à ses produits.
Alternatives et exceptions	Si les informations obligatoires du produit sont incomplètes ou invalides, le système demande leur correction. Si l'Administrateur Banque rejette la demande de publication, le produit n'est pas publié dans la marketplace concernée. Si le partenariat ou le contrat avec la banque n'est plus actif, la publication du produit n'est pas autorisée.

6.6. Cas d'utilisation détaillée : Déposer et traiter une demande de financement

Ce cas d'utilisation décrit le processus de demande de financement, depuis la constitution du dossier par le client jusqu'à son traitement par l'Administrateur Banque. La figure et le tableau associés présentent les acteurs concernés, les principales étapes du processus ainsi que les différentes décisions pouvant être prises sur le dossier.

Parcours client et demande de financement

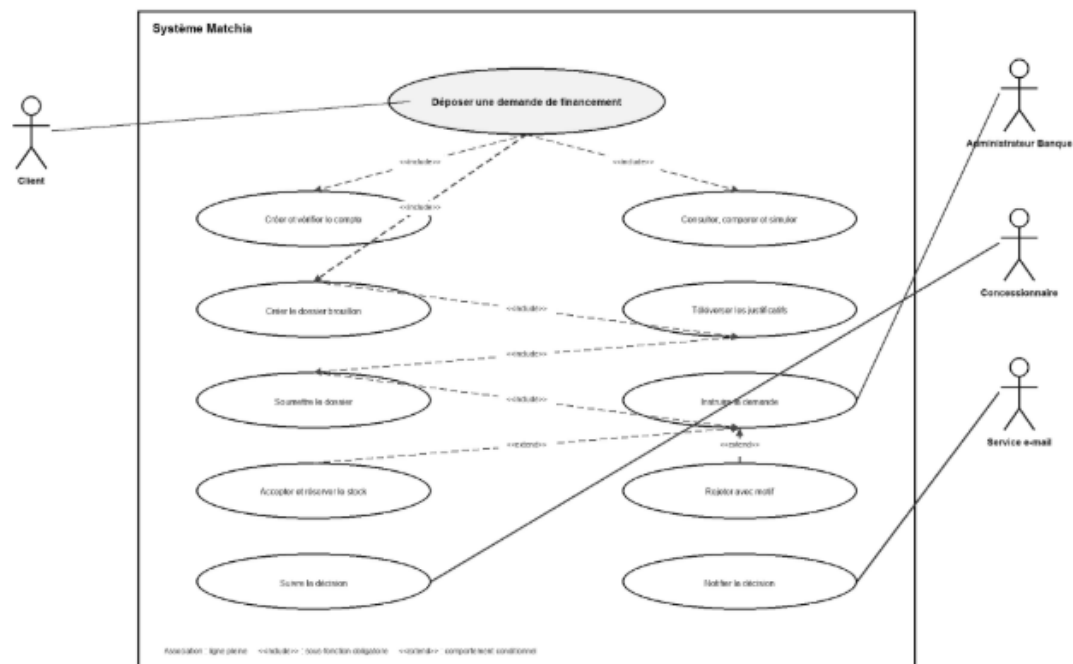


Figure 3.13 — Diagramme de cas d'utilisation détaillé de la demande de financement

Le tableau suivant présente le scénario du cas d'utilisation **UC-06 — Déposer et traiter une demande de financement**.

Tableau 3.9 — Description du scénario UC-06

Élément	Description
Acteur principal	Client
Acteur secondaire	Administrateur Banque, Concessionnaire et service e-mail
Objectifs	Permettre au client de constituer et de soumettre une demande de financement afin d'obtenir une décision de la banque concernée.
Préconditions	Le client possède un compte vérifié et est connecté. Le produit sélectionné est disponible et publié dans la marketplace.
Postconditions	La demande est enregistrée avec une décision d'acceptation ou de rejet. Le client est informé de la décision et, en cas d'acceptation d'une demande concernant un produit concessionnaire, le stock du produit est mis à jour.
Scénario nominal	- Le client consulte le produit souhaité et peut utiliser le comparateur ou le simulateur lorsqu'ils sont disponibles. - Il démarre une demande de financement.

	3. Il renseigne les informations nécessaires à la constitution du dossier. 4. Il dépose les pièces justificatives demandées puis soumet le dossier. 6. Le système enregistre la demande avec le statut « en attente » et notifie l'Administrateur Banque. 7. L'Administrateur Banque consulte le dossier et les documents transmis. 8. Il accepte la demande. 9. Le système met à jour le statut du dossier et informe le client par e-mail et par une notification dans votre espace. 10. Le stock du produit correspondant est mis à jour et le concessionnaire peut consulter le dossier associé à son produit.
Alternatives et exceptions	Si des informations ou des pièces obligatoires sont manquantes, la soumission n'est pas autorisée. Si l'Administrateur Banque rejette la demande, il renseigne le motif du rejet et le client en est informé par e-mail. Le concessionnaire peut consulter les demandes associées à ses produits, mais ne participe pas à la décision de financement.

7. Conclusion

Chapitre 3 : Architecture et Conception

1. Introduction

Ce chapitre présente les fondements architecturaux et techniques de la plateforme Matchia. Il introduit d'abord le modèle SaaS et le principe de la multi-tenancy, puis examine les principales stratégies d'isolation des données afin de justifier le choix retenu pour le projet. Il décrit ensuite l'architecture globale de la solution à travers ses vues logique et physique, avant de présenter l'environnement de développement ainsi que les technologies mobilisées pour sa réalisation.

2. Architecture SaaS et de la multi-tenant

2.1. Présentation du SaaS

Le **Software as a Service (SaaS)** est un modèle de distribution dans lequel une application est accessible en ligne sous forme de service, généralement hébergée dans le Cloud. Le fournisseur prend en charge l'hébergement, la maintenance, les mises à jour et l'exploitation technique, tandis que les utilisateurs accèdent à la solution à distance, souvent via un navigateur web et selon un modèle d'abonnement sans avoir à gérer directement l'installation, l'infrastructure ou la maintenance technique[3].

2.2. Principe de multi-tenant

Le multi-tenant est un principe d'architecture fréquemment associé au modèle SaaS. Elle permet à plusieurs clients ou organisations d'utiliser la même application, tout en conservant chacun un environnement logique qui lui est propre. Chaque client, appelé **tenant**, partage ainsi le même socle technique, tandis que ses données restent isolées de celles des autres tenants.

Ce principe repose sur deux notions essentielles : la mutualisation des ressources et l'isolation des données. L'application est développée et maintenue une seule fois, puis mise à disposition de plusieurs tenants. Des mécanismes de contrôle permettent ensuite d'identifier le tenant concerné et de garantir que chaque utilisateur accède uniquement aux données et aux fonctionnalités relevant de son propre périmètre. Cette approche permet ainsi de combiner centralisation, personnalisation et séparation logique au sein d'une même plateforme.

2.3. Principales stratégies multi-tenant

Plusieurs stratégies peuvent être adoptées pour mettre en œuvre une architecture multi-tenant. Elles se distinguent principalement par la manière dont les données des différents tenants sont stockées, isolées et administrées [4].

1. Base de données dédiée par tenant

Dans cette stratégie, chaque tenant dispose de sa propre base de données. Les données de chaque organisation sont donc physiquement séparées de celles des autres tenants, ce qui offre une isolation maximale des données, ce qui renforce la sécurité et la confidentialité, et facilite les sauvegardes ou restaurations propres à un client. En revanche, elle engendre un coût d'infrastructure élevé, puisqu'il faut gérer autant de bases que de clients, et devient plus difficile à maintenir et à faire évoluer à mesure que le nombre de tenants augmente.

2. Schéma dédié par tenant

Cette approche repose sur une base de données commune, mais chaque tenant possède son propre schéma. Les données restent ainsi séparées de manière structurelle, tout en partageant la même infrastructure de base de données. Cette stratégie constitue un compromis entre isolation et mutualisation, car elle réduit le nombre de bases à administrer. Toutefois, la gestion des schémas, des migrations et des évolutions peut devenir complexe lorsque le nombre de tenants devient important.

3. Base et schéma partagés

Dans cette stratégie, tous les tenants utilisent la même base de données et le même schéma. Les données sont stockées dans des tables communes et sont distinguées à l'aide d'identifiants permettant de déterminer à quel tenant chaque donnée appartient. L'isolation est donc principalement assurée au niveau logique et applicatif. Cette approche facilite la mutualisation des ressources, réduit les coûts d'infrastructure et simplifie la maintenance globale. Elle exige toutefois des contrôles rigoureux afin de garantir qu'un tenant ne puisse jamais accéder aux données d'un autre.



Figure 3: Principales stratégies d'architecture multi-tenant

2.4. Choix d'architecture

Après l'analyse des différentes stratégies de multi-tenant, le choix d'une base de données unique et partagée avec une isolation logique des données constitue la solution la plus adaptée à notre projet. Cette architecture permet de centraliser les données des différentes banques dans une même infrastructure, tout en maintenant une séparation stricte entre les tenants grâce à des mécanismes d'identification, de filtrage et de contrôle d'accès. Elle simplifie l'administration de la plateforme, facilite l'intégration de nouvelles banques et réduit la complexité liée à la gestion de plusieurs bases indépendantes. Ce choix offre ainsi un bon compromis entre mutualisation, maintenabilité, évolutivité et maîtrise des coûts, tout en exigeant une gestion rigoureuse de l'isolation des données au niveau applicatif.

3. Architecture globale

Après avoir défini le modèle SaaS multi-tenant adopté pour Matchia, il convient de présenter l'organisation générale de la solution. L'architecture retenue permet de séparer clairement l'interface utilisateur, la logique métier, l'accès aux données utilisés par la plateforme.

Matchia repose sur une architecture monolithique en couches avec un frontend séparé. Le frontend, développé avec React et TypeScript, constitue la couche d'interaction avec les utilisateurs. Il communique avec le backend Spring Boot à travers des API REST. Le backend centralise l'ensemble des traitements métier de la plateforme et assure l'accès aux données stockées dans PostgreSQL.

Cette organisation s'inspire également des principes du modèle MVC, notamment par la séparation entre la vue, les contrôleurs et les composants représentant le modèle. Toutefois, dans Matchia, la vue est assurée par un frontend React indépendant, tandis que le backend est structuré principalement autour des couches Controller, Service et Repository.

Afin de mieux représenter cette organisation, nous distinguons deux vues complémentaires : l'architecture logique, qui décrit la structuration interne des composants logiciels, et l'architecture physique, qui présente leur organisation au niveau de l'environnement d'exécution.

3.1. Architecture logique

L'architecture logique décrit la manière dont les différents composants logiciels de Matchia sont organisés ainsi que les interactions entre les différentes couches de l'application.

Dans Matchia, cette architecture suit une organisation en couches. Chaque couche possède une responsabilité précise et communique principalement avec la couche qui lui est directement associée.

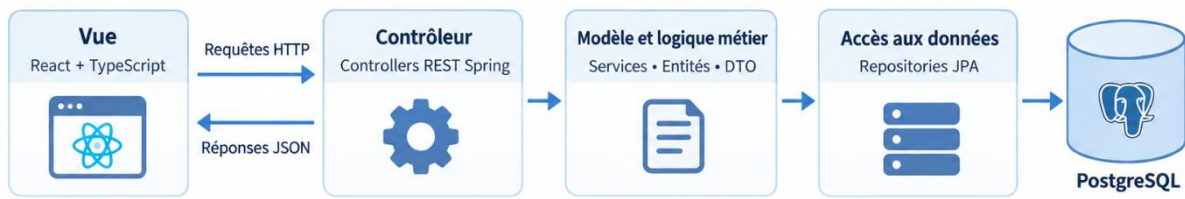


Figure X.X — Architecture logique de la plateforme Matchia

La première couche correspond à la vue, développée par React et TypeScript, qui fournit les différentes interfaces destinées aux utilisateurs de la plateforme. À partir de ces interfaces, les actions effectuées par l'utilisateur sont transformées en requêtes HTTP envoyées au backend via des API REST, tandis que les réponses sont principalement retournées au format JSON. Ces requêtes sont reçues par les contrôleurs REST, qui jouent le rôle d'intermédiaire entre le frontend et la logique applicative. Ils analysent les demandes reçues puis transmettent le traitement aux services métier, chargés d'appliquer les règles fonctionnelles propres à Matchia. Cette couche métier s'appuie également sur les entités et les DTO, utilisés respectivement pour représenter les données de l'application et organiser les informations échangées entre les différentes couches. Lorsque le traitement nécessite un accès aux données, les services sollicitent les Repositories JPA, qui prennent en charge les opérations de persistance. Ces repositories communiquent ensuite avec la base de données PostgreSQL, dans laquelle sont stockées les informations de la plateforme. Cette base est commune aux différents tenants, avec une isolation logique assurée au niveau applicatif.

Cette organisation assure une séparation claire des responsabilités et contribue à rendre l'application plus lisible, maintenable et évolutive.

3.2. Architecture physique

L'architecture physique de Matchia décrit la mise en œuvre concrète de la solution au niveau de son environnement d'exécution. Elle met en évidence les différents composants techniques de la plateforme, notamment le frontend, le backend, la base de données et les services externes, ainsi que les protocoles et mécanismes de communication assurant leurs échanges de manière sécurisée et cohérente.

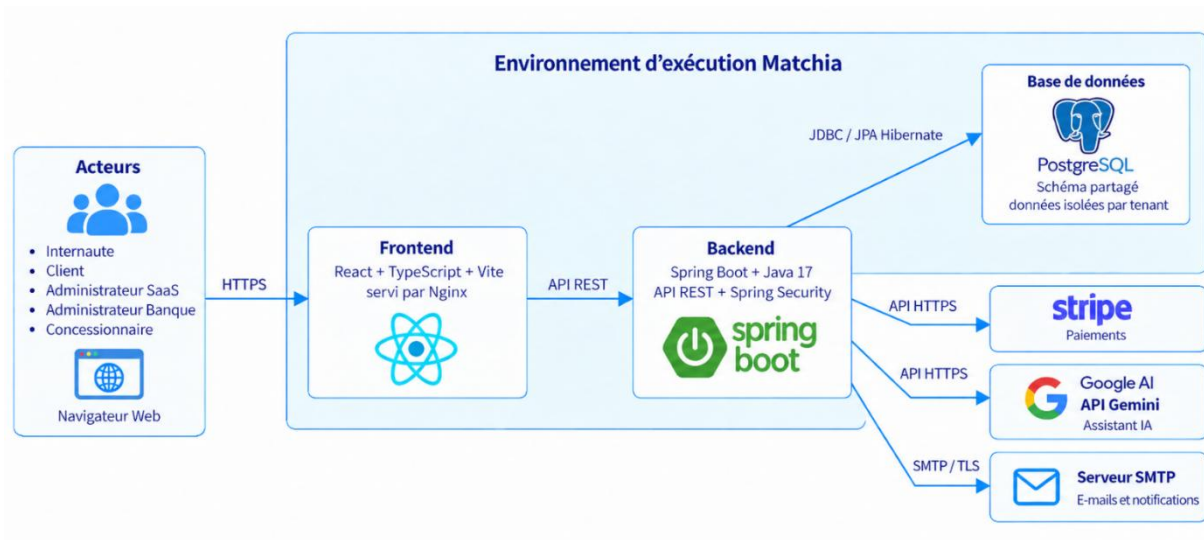


Figure 3.x — Architecture physique de la plateforme Matchia

Les différents acteurs de Matchia, comprennent l'internaute, le client, l'Administrateur SaaS, l'Administrateur Banque et le concessionnaire, accèdent à la plateforme à travers un navigateur Web. Les requêtes sont d'abord prises en charge par le frontend, développé avec React, TypeScript et Vite et servi par Nginx, qui assure l'affichage des interfaces ainsi que la gestion des interactions avec les utilisateurs. Lorsqu'une opération nécessite un traitement côté serveur, le frontend transmet la requête au backend à travers des API REST. Développé avec Spring Boot et Java 17 celui-ci constitue le cœur applicatif de Matchia : il centralise la logique métier et assure le traitement des différentes fonctionnalités de la plateforme. Il s'appuie notamment sur Spring Security pour gérer l'authentification, l'autorisation et le contrôle des accès selon le rôle de chaque utilisateur. Lorsque le traitement nécessite la consultation ou la modification de données, le backend communique avec la base PostgreSQL à l'aide de JPA/Hibernate et du pilote JDBC. La plateforme adopte une base et un schéma partagés entre les différentes banques. L'isolation des données est donc assurée logiquement par le backend, qui filtre les informations selon le tenant concerné afin qu'une banque ne puisse pas accéder aux données d'une autre banque. Le backend peut également communiquer avec plusieurs services externes selon la fonctionnalité exécutée : Stripe pour la gestion des paiements, l'API Gemini de Google AI pour les fonctionnalités d'assistance intelligente et un serveur SMTP pour l'envoi des e-mails et des notifications.

4. Environnement de développement et technologies utilisées

4.1. Environnement matériel

Pour développer notre solution, nous avons utilisé un ordinateur dont les principales caractéristiques matérielles sont présentées dans le tableau suivant.

Processeur	11th Gen Intel(R) Core(TM) i5-11300H @ 3.10GHz (3.11 GHz)
RAM	16,0 Go
Système d'exploitation	Windows 11

Tableau : Caractéristiques de l'environnement matériel

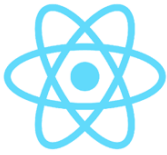
4.2. Environnement logiciel

Pour assurer le développement et les tests de notre solution, nous avons utilisé plusieurs outils logiciels présentés ci-dessous.

 Visual Studio Code	Visual Studio Code est un éditeur de code source développé par Microsoft. Léger et extensible, il prend en charge plusieurs langages de programmation.
 IntelliJ IDEA	IntelliJ IDEA est un environnement de développement intégré développé par JetBrains. Il offre de nombreux outils facilitant le développement d'applications Java
 Postman	Postman est une plateforme dédiée au développement et au test des API. Elle permet d'envoyer des requêtes HTTP et de vérifier les réponses retournées par le serveur.

 pgAdmin	pgAdmin est une plateforme open source d'administration spécialement conçue pour PostgreSQL. Elle permet notamment de créer, consulter et administrer les bases de données.
 DBeaver	DBeaver est un outil de gestion de bases de données permettant de consulter les données et d'exécuter des requêtes SQL à travers une interface graphique.
 GitHub	GitHub est une plateforme d'hébergement et de gestion de code source basée sur Git. Elle facilite le suivi des versions et le travail collaboratif.

4.3. Technologies utilisées

 React	React est une bibliothèque JavaScript permettant de construire des interfaces utilisateur à partir de composants réutilisables.
 Spring Boot	Spring Boot est un framework Java basé sur l'écosystème Spring. Il fournit un ensemble de bibliothèques facilitant la création rapide et la gestion simplifiée des projets.

 PostgreSQL	PostgreSQL est un système de gestion de base de données relationnelle open source permettant de stocker, organiser et manipuler les données de manière fiable et sécurisée. Il utilise principalement le langage SQL et prend en charge des fonctionnalités avancées.
 Docker	Docker est une plateforme de conteneurisation permettant d'emballer une application avec ses dépendances dans des conteneurs. Elle facilite ainsi l'exécution de l'application dans différents environnements.
 Docker Compose	Docker Compose est un outil de l'écosystème Docker permettant de définir et de lancer plusieurs conteneurs à partir d'un seul fichier de configuration. Il est utilisé pour orchestrer les différents services nécessaires au fonctionnement d'une application.
 Jenkins	Jenkins est un outil open source d'automatisation qui sert principalement à mettre en place l'intégration continue et le déploiement continu (CI/CD) dans le développement logiciel.
 SonarQube	SonarQube est une plateforme d'analyse de la qualité du code développée par SonarSource. Elle permet de détecter les bugs, vulnérabilités et les mauvaises pratiques.
 Docker Hub	Docker Hub est un registre en ligne permettant de stocker, gérer et distribuer des images de conteneurs Docker.

5. Conclusion

En conclusion, ce chapitre a permis de présenter les fondements architecturaux et techniques de Matchia. Il a introduit le modèle SaaS et le principe de la multi-tenancy, comparé les principales stratégies d'isolation des données et justifié le choix d'une base et d'un schéma partagés avec une isolation logique par tenant. Il a également décrit l'architecture monolithique en couches de la plateforme, à travers ses vues logique et physique, ainsi que l'environnement de développement et les technologies retenues pour sa réalisation. Ces choix constituent un socle cohérent, maintenable et évolutif pour la mise en œuvre des fonctionnalités présentées dans les chapitres suivants.

Chapitre 4 : Mise en œuvre de la plateforme SaaS et gestion du cycle de vie des marketplaces bancaires

1. Introduction

Ce chapitre est consacré à la réalisation de la plateforme **Matchia** et à la mise en œuvre de ses principales fonctionnalités. Il présente le parcours complet de création d'une marketplace bancaire, depuis la soumission de la demande jusqu'à son activation et sa configuration par l'Administrateur Banque. Les différents espaces et workflows sont décrits afin de mettre en évidence le fonctionnement concret de la solution et les interactions entre ses principaux acteurs.

2. Authentification, autorisation et contexte multi-tenant

Avant de présenter les différents parcours fonctionnels, il est nécessaire d'introduire les mécanismes qui assurent la sécurisation des accès et la gestion du contexte multi-tenant.

L'accès aux espaces privées repose sur un système d'authentification permettant notamment la connexion, la déconnexion, le renouvellement de la session ainsi que la réinitialisation du mot de passe. Une fois authentifié, l'utilisateur est dirigé vers l'espace correspondant à son rôle.

La plateforme distingue principalement quatre rôles : Administrateur SaaS, Administrateur Banque, Concessionnaire et Client. Chaque rôle dispose d'un périmètre fonctionnel spécifique et ne peut accéder qu'aux fonctionnalités qui lui sont autorisées.

L'Administrateur SaaS dispose d'une vision globale de la plateforme et en assure la supervision générale. L'Administrateur Banque gère uniquement les ressources et services liés à sa propre marketplace. Les Concessionnaires accèdent aux fonctionnalités associées à leurs partenariats bancaires, tandis que les Clients utilisent les services proposés par la marketplace correspondant à leur banque.

Dans le cadre de l'architecture multi-tenant, les utilisateurs de chaque banque sont également associés au contexte de leur établissement. Ce mécanisme permet à plusieurs banques d'utiliser le même socle applicatif tout en conservant leurs propres utilisateurs, contenus, produits, configurations, stores et modules.

Ces mécanismes constituent la base de sécurité et d'isolation sur laquelle reposent les différents workflows présentés dans la suite de ce chapitre.

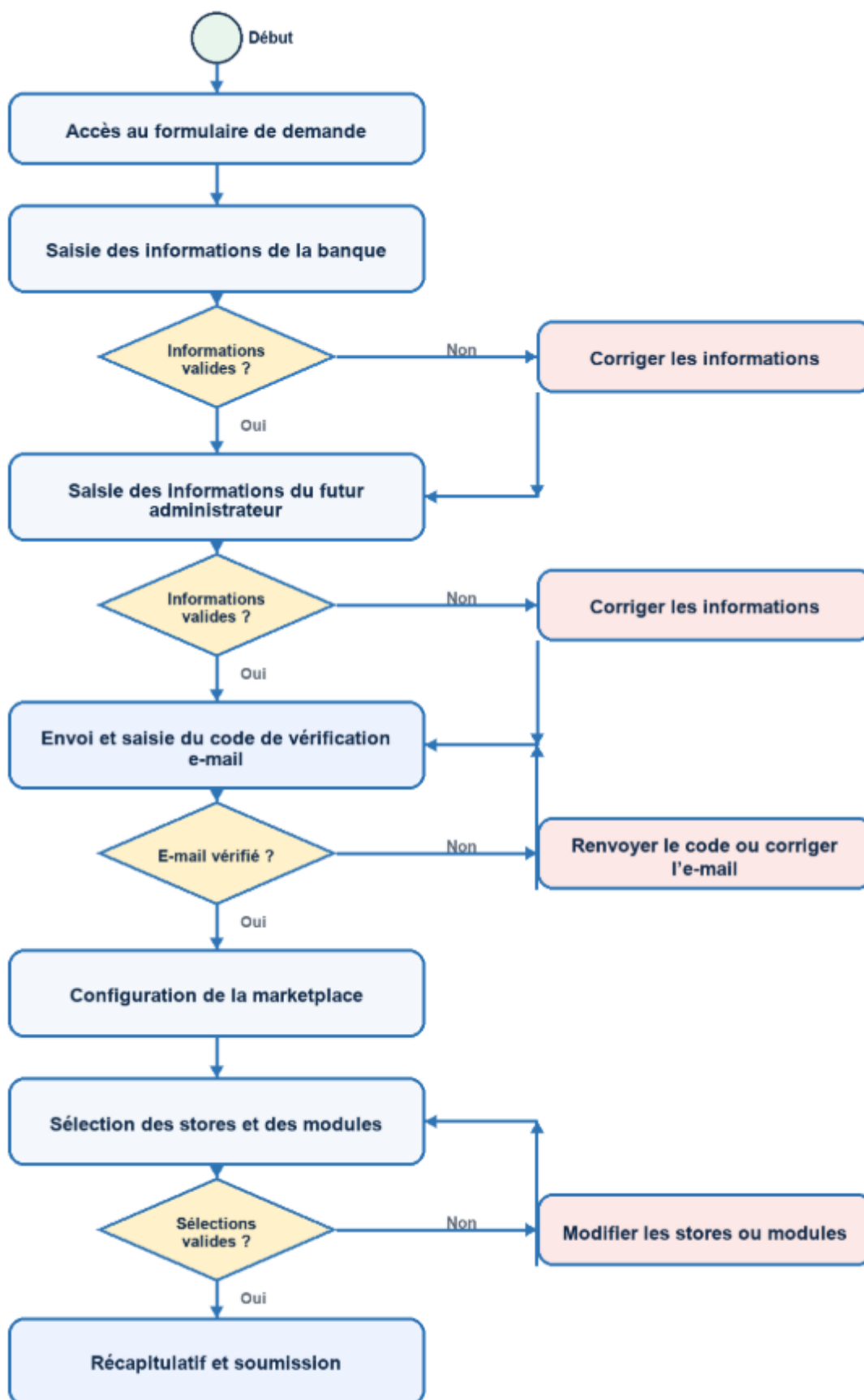
3. Demande de création d'une marketplace bancaire

La première étape du processus consiste pour une banque à soumettre une demande d'adhésion à Matchia. Cette démarche est accessible depuis l'espace public de la plateforme et permet au demandeur de renseigner progressivement les informations nécessaires à la création de sa future marketplace.

Capture accueil public

Afin de faciliter la saisie et de limiter les demandes incomplètes, le formulaire est organisé en plusieurs étapes successives. Le parcours comprend les informations de la banque, les coordonnées du futur administrateur, la configuration initiale de la marketplace, la sélection des stores et modules, puis la validation finale de la demande.

Le processus général peut être représenté par le diagramme d'activité suivant :



3.1. Renseignement des informations de la banque

La première étape permet de recueillir les informations nécessaires à l'identification de l'établissement bancaire. Le demandeur renseigne notamment le nom de la banque, son adresse électronique, son numéro de téléphone, son site web, une description ainsi que certaines informations complémentaires relatives à l'établissement.

Figure 4.2 — Saisie des informations de la banque

La figure présente le premier écran du formulaire avec les différentes informations demandées ainsi que la zone de téléversement du logo.

3.2. Informations du futur Administrateur Banque et vérification de l'adresse e-mail

La deuxième étape est consacrée au futur administrateur de la marketplace. Le formulaire recueille notamment son nom, son prénom, son adresse électronique, son numéro de téléphone ainsi que son image de profil.

Une vérification de l'adresse électronique est intégrée à ce parcours. Après validation des informations saisies, un **code de vérification** est envoyé à l'adresse indiquée. Le demandeur doit saisir ce code dans le formulaire afin de confirmer que l'adresse lui appartient réellement.

Cette vérification joue un rôle important, car l'adresse validée sera ensuite utilisée dans le processus de communication avec la banque, notamment pour l'envoi des informations relatives au traitement de la demande, au paiement et à l'accès au Back-office Banque.

Figure 4.3 — Vérification de l'adresse électronique du futur Administrateur Banque

La figure doit présenter l'interface de saisie du code ainsi que la confirmation de la vérification de l'adresse électronique. Les informations personnelles utilisées dans les captures doivent être anonymisées.

3.3. Configuration initiale de la marketplace, sélection des stores et modules

Cette étape permet notamment de définir certains éléments propres à l'identité de la marketplace, notamment son identifiant d'accès (*slug*), ses couleurs de personnalisation ainsi que la bannière affichée sur la page d'accueil.

La banque sélectionne ensuite les stores qu'elle souhaite intégrer à sa future marketplace. Chaque store représente un univers fonctionnel dans lequel pourront être proposés des produits et services spécifiques.

Une fois un store sélectionné, la plateforme affiche les modules qui lui sont associés. La banque peut alors choisir les fonctionnalités adaptées à ses besoins et à son offre.

Figure 4.4 — Sélection des stores de la marketplace

La figure présente les stores disponibles et la sélection effectuée par la banque.

3.4. Récapitulatif et soumission de la demande

La dernière étape du formulaire affiche un récapitulatif permettant au demandeur de vérifier l'ensemble des données renseignées avant la validation définitive de sa demande. Cette synthèse regroupe les informations de la banque, les coordonnées du futur administrateur Banque, les éléments de configuration de la marketplace, les stores sélectionnés, les modules associés à chaque store ainsi que les informations relatives à l'offre choisie.

Après la soumission, une notification est créée à l'intention de l'Administrateur SaaS afin de signaler l'arrivée d'une nouvelle demande. Parallèlement, un e-mail de confirmation est envoyé au contact de la banque.

Figure 4.7 — Confirmation de la soumission de la demande

Ces figures permettent respectivement de présenter la synthèse du formulaire et le message confirmant l'enregistrement de la demande.

4. Back-office SaaS : administration et supervision de la plateforme

Après la soumission d'une demande, son traitement est assuré par l'Administrateur SaaS.

Le Back-office SaaS constitue l'espace central de gouvernance de la plateforme. Il regroupe les fonctionnalités nécessaires au pilotage des ressources partagées, au suivi des demandes et au contrôle du cycle de vie des services proposés aux banques.

4.1. Tableau de bord

Le tableau de bord fournit une vue globale de l'activité de la plateforme à travers différents indicateurs permettant de suivre notamment les banques, les demandes reçues, les abonnements et les principales informations. Il permet ainsi à l'Administrateur SaaS d'identifier rapidement les éléments nécessitant son intervention.

Figure 4.8 — Tableau de bord de l'Administrateur SaaS

4.2. Gestion des stores et des modules

Cette rubrique permet d'administrer le catalogue des stores et des modules proposés aux banques. L'Administrateur SaaS peut consulter les univers fonctionnels disponibles, organiser les modules associés et définir les éléments qui pourront être sélectionnés lors de la création d'une marketplace. Cette gestion centralisée garantit la cohérence des services mis à disposition des différents tenants.

Figure 4.8 — Gestion stores et modules

4.3. Gestion des demandes de marketplace

La rubrique Demandes regroupe l'ensemble des demandes liées à la création ou à l'évolution des services d'une marketplace. Elle permet de les consulter, de les analyser et de les traiter de manière organisée tout en assurant leur suivi et leur traçabilité.

4.3.1. Traitement d'une demande de marketplace

L'Administrateur SaaS dispose d'une interface permettant de consulter les demandes reçues et d'accéder à leur détail.

La fiche d'une demande présente les informations de la banque, celles du futur administrateur, la configuration souhaitée ainsi que les stores et modules sélectionnés.

Après vérification, deux décisions sont possibles : **approbation** ou **rejet** :

- En cas de rejet, l'administrateur peut renseigner un motif permettant d'expliquer la décision ainsi qu'un e-mail transmis au demandeur.
- En cas d'approbation, le processus se poursuit vers l'étape de paiement.

Figure 4.9 — Liste des demandes de marketplace dans le Back-office SaaS

Figure 4.10 — Consultation du détail d'une demande

Le processus décisionnel peut également être représenté à travers un diagramme d'activité(NAJA)

Figure 4.11 — Diagramme d'activité du traitement d'une demande par l'Administrateur SaaS

4.3.2. Approbation et déclenchement du paiement

Lorsqu'une demande est approuvée, Matchia prépare les éléments nécessaires à la future activation de la banque et de sa marketplace. Toutefois, ces éléments restent inactifs tant que le paiement n'a pas été confirmé.

À cette étape, un e-mail contenant les instructions de paiement ainsi que le lien correspondant est envoyé au contact de la banque.

4.3.3. Paiement sécurisé par Stripe et activation de la marketplace

Le paiement constitue une étape déterminante dans le cycle de vie de la demande. Il est intégré à la plateforme au moyen de Stripe, service de paiement en ligne. Lorsque le futur Administrateur Banque accède à l'interface de paiement, il renseigne les informations requises afin d'effectuer la transaction. Une fois le paiement validé et confirmé par le prestataire, Matchia met à jour les ressources associées : la banque et sa marketplace sont activées, l'abonnement passe à l'état actif et le compte de l'Administrateur Banque est autorisé à accéder à son espace de gestion.

La plateforme génère ensuite les informations nécessaires à la première connexion et envoie au responsable de la banque un e-mail contenant ses identifiants d'accès au Back-office Banque.

Le workflow complet est représenté par le diagramme de séquence suivant.

Figure 4.12 — Diagramme de séquence de l'approbation, du paiement et de l'activation

Figure 4.13 — Email contenant les instructions de paiement

Figure 4.14 — Confirmation du paiement

Figure 4.15 — Activation de la marketplace et de l'abonnement

Figure 4.16 — Email d'accès au Back-office Banque

4.4. Gestion des banques

La rubrique Banques permet à l'Administrateur SaaS de consulter les établissements enregistrés sur la plateforme et de suivre leur état d'activation. Elle offre une vue sur les informations propres à chaque banque.

Figure 4.16 — Liste des Banques

4.5. Gestions des utilisateurs et des rôles

Cette fonctionnalité permet de consulter les utilisateurs de la plateforme et d'administrer les rôles attribués. Elle contribue à appliquer les règles d'autorisation : chaque utilisateur accède exclusivement aux espaces et aux opérations correspondant à son profil.

Figure 4.16 — Liste des utilisateurs

4.6. Gestion des concessionnaires

L'Administrateur SaaS peut consulter et traiter les demandes d'inscription des concessionnaires en vérifiant les informations et les documents transmis avant de les accepter ou de les refuser. Cette rubrique permet également de gérer les comptes concessionnaires actifs et de consulter leurs informations.

Figure 4.16 — Liste des concessionnaires

4.7. Gestion des marketplaces

La rubrique Marketplace permet de visualiser les marketplaces créées pour les banques et d'en suivre le statut. Elle constitue un point de contrôle sur les espaces publics associés aux tenants et sur leur disponibilité après validation de l'abonnement.

Figure 4.16 —liste marketplace

4.8. Gestion du contenu des marketplaces

La gestion du contenu permet de gérer les éléments visuels et textuels affichés au sein des marketplaces. L'Administrateur SaaS peut superviser les contenus disponibles tout en permettant à chaque banque de personnaliser les éléments propres à sa marketplace.

Figure 4.16 — Contenu marketplace

4.9. Gestion des abonnements

Cette rubrique permet de suivre les abonnements associés aux marketplaces. Chaque abonnement correspond aux services sélectionnés par la banque ainsi qu'à leur période de validité.

Lors de la création initiale, l'abonnement reste en attente du paiement. Une fois celui-ci confirmé, il devient actif et les stores et modules associés peuvent être exploités.

Le Back-office SaaS permet également de suivre les abonnements, leurs périodes de validité et leurs statuts. Des mécanismes permettent de surveiller leur échéance et d'informer les acteurs concernés lorsqu'un abonnement approche de sa date d'expiration.

Cette gestion permet d'assurer que seuls les services disposant d'un abonnement valide restent accessibles

Figure 4.16 — liste des abonnements

4.10. Audit et logs

La rubrique Audit & Logs fournit une consultation des événements importants réalisés sur la plateforme. Elle contribue à la traçabilité des opérations d'administration, des actions liées aux demandes et des opérations de paiement, ce qui facilite le suivi et l'analyse des incidents.

Figure 4.16 — liste des abonnements

5. Back-office Banque : administration de la marketplace

Une fois connecté, l'Administrateur Banque dispose d'un espace exclusivement associé à son établissement. Le contexte tenant garantit que les informations affichées correspondent uniquement à sa marketplace.

Le tableau de bord fournit une synthèse de l'activité de la banque et constitue le point d'entrée vers les différentes fonctionnalités administratives.

Figure 4.17 — Tableau de bord de l'Administrateur Banque

L'espace bancaire permet à l'Administrateur Banque de gérer la personnalisation de sa marketplace, les stores et les modules activés, les produits et les contenus publiés, ainsi que la configuration du simulateur. Il lui offre également la possibilité d'administrer les concessionnaires et les partenariats, de consulter les clients rattachés à sa banque, de traiter les demandes de financement, de gérer les abonnements et de consulter les notifications reçues.

5.1. Personnalisation de la marketplace

Chaque banque peut personnaliser l'identité visuelle et le contenu de sa marketplace sans modifier le socle applicatif partagé. L'Administrateur Banque dispose ainsi d'outils lui permettant de définir les couleurs principales, le logo, les bannières, les textes d'accueil, les contenus propres à la marketplace ainsi que les contenus associés à chaque store.

Cette personnalisation permet à plusieurs établissements d'utiliser une même application tout en proposant une expérience adaptée à leur identité et à leur offre.

Figure 4.18 — Interface de personnalisation de la marketplace

Figure 4.19 — Gestion des contenus et des bannières

5.2. Gestion des stores, modules et paramètres

La banque peut consulter les stores qui lui sont attribués dans le cadre de son abonnement. Chaque store dispose de modules fonctionnels qui lui sont propres. L'Administrateur Banque peut visualiser les fonctionnalités disponibles, gérer l'activation des modules autorisés et configurer leurs paramètres selon les besoins de sa marketplace.

Les paramètres configurables sont notamment utilisés par le simulateur. La banque peut définir les valeurs nécessaires aux calculs de financement. Ces valeurs sont ensuite exploitées par la marketplace publique lors de l'exécution d'une simulation.

Figure 4.20 — Stores associés à la marketplace

Figure 4.21 — Gestion des modules et de leurs paramètres

5.3. Renouvellement et l'ajout des services

L'Administrateur Banque peut consulter les informations relatives à son abonnement, notamment son statut, sa période de validité, les services associés et son historique.

Lorsque la banque souhaite conserver les mêmes services après leur échéance, elle peut initier une demande de renouvellement. Lorsqu'elle souhaite intégrer de nouveaux stores ou modules, elle soumet une demande d'extension d'abonnement.

Dans les deux cas, la demande est transmise à l'espace SaaS, où elle est examinée et approuvée. Elle se poursuit ensuite par l'étape de paiement ; après confirmation de la transaction, l'abonnement est mis à jour et les nouveaux services sont activés.

Figure 4.22 — Diagramme d'activité du renouvellement et de l'extension d'un abonnement

Figure 4.23 — Consultation de l'abonnement et de son historique

Figure 4.24 — Soumission d'une demande de renouvellement ou d'extension

6. Marketplace publique

Une fois la marketplace active et configurée, elle devient accessible depuis son espace public.

L'interface reprend les paramètres et éléments de personnalisation définis par la banque. Elle présente notamment le logo, les couleurs, les bannières, les contenus ainsi que les stores disponibles.

Figure 4.25 — Page d'accueil d'une marketplace bancaire

La marketplace est organisée autour de plusieurs **stores**, chacun correspondant à un univers fonctionnel spécifique. Chaque store dispose de ses propres modules, définis selon la configuration de la banque. Ces modules déterminent les fonctionnalités accessibles dans le

store et peuvent être activés ou désactivés en fonction des services souscrits et des besoins de la marketplace.

Figure 4.26 — Consultation d'un store

7. Conclusion

En conclusion, ce chapitre a permis de présenter la mise en œuvre de la plateforme à travers ses principaux parcours fonctionnels. Il a détaillé le cycle de vie d'une marketplace, depuis la soumission de la demande jusqu'à son activation, sa personnalisation et son exploitation. L'ensemble des fonctionnalités réalisées constitue ainsi une base concrète pour répondre aux besoins des différents acteurs et assurer le bon fonctionnement de la solution.

Chapitre 5 : Mise en œuvre des parcours des concessionnaires et des clients, et intégration des assistants conversationnels

1. Introduction

Ce chapitre a présenté les différentes fonctionnalités mises en œuvre pour les Concessionnaires et les Clients, depuis la gestion des produits et des partenariats jusqu'au processus de financement, ainsi que l'intégration de l'assistant IA et du chatbot.

2. Parcours concessionnaires

Le concessionnaire constitue un acteur important de l'écosystème Matchia puisqu'il peut proposer ses produits auprès de plusieurs banques. Afin de faciliter cette collaboration, la plateforme lui fournit un espace centralisé à partir duquel il peut gérer son activité sans avoir à créer un compte distinct pour chaque banque partenaire. Cette approche permet de conserver un point de gestion unique tout en maintenant la séparation des partenariats et des publications selon chaque banque.

2.1. Inscription et validation du concessionnaire

Le parcours débute par une demande d'inscription accessible depuis l'espace public. Le concessionnaire renseigne les informations relatives à son entreprise, ses coordonnées, le store concerné ainsi que les données nécessaires à l'identification de son activité. Il doit également transmettre les documents justificatifs requis, afin de permettre la vérification de son identité, de la conformité de son activité et de son éligibilité avant le traitement de la demande.

Une fois le formulaire soumis, la demande est enregistrée puis transmise à l'Administrateur SaaS pour traitement. Après analyse, elle peut être approuvée ou rejetée. En cas d'approbation, le système crée le compte du concessionnaire et lui donne accès à son espace personnel. Un e-mail contenant ses identifiants de connexion lui est alors envoyé. En cas de rejet, un e-mail précisant le motif de la décision est transmis au demandeur.

Figure 5.1 — Diagramme d'activité du parcours d'inscription et de validation d'un concessionnaire

Capture 5.1 — Formulaire d'inscription du concessionnaire

Cette capture présente le formulaire permettant au concessionnaire de renseigner les informations nécessaires à la création de sa demande.

Capture 5.2 — Gestion des demandes concessionnaires par l'Administrateur SaaS

Cette capture illustre l'interface permettant au SaaS de consulter, filtrer et traiter les demandes reçues.

2.2. Partenariats et contrats

Une fois le compte concessionnaire validé, la collaboration avec une banque est organisée à travers un mécanisme de partenariat. Dans Matchia, le partenariat constitue le lien entre un concessionnaire, une banque et un store donné. Cette modélisation permet à un même concessionnaire de collaborer avec plusieurs banques tout en conservant un compte unique.

La demande de partenariat peut être initiée dans les deux sens. Depuis son espace, le concessionnaire sélectionne la banque auxquels il souhaite proposer sa collaboration ; la banque destinataire peut ensuite consulter la demande et l'approuver ou la rejeter. Inversement, l'Administrateur Banque peut sélectionner un concessionnaire disponible et lui adresser une demande de partenariat. Le concessionnaire reçoit alors cette demande dans son espace et peut à son tour l'accepter ou la rejeter.

Le contrat définit les conditions de collaboration entre la banque et le concessionnaire. Il précise notamment la période de validité, les commissions éventuellement applicables, les conditions contractuelles ainsi que les modalités de résiliation. Son cycle de vie comprend les états d'attente d'acceptation, d'activation, d'annulation, d'expiration ou de résiliation. L'activation intervient après l'acceptation du concessionnaire et la validation finale de la banque.

Capture 5.3 — partenariat Concessionnaire vers banque

Capture 5.3 — partenariats Banque vers Concessionnaire

Figure 5.2 — Cycle de vie d'un partenariat, d'un contrat et d'une publication

La figure permet de visualiser l'enchaînement entre la création du partenariat, la formalisation du contrat et l'autorisation de publication du produit.

Capture 5.3 — Gestion des partenariats et des contrats

Cette interface permet au concessionnaire de consulter ses relations avec les différentes banques ainsi que les contrats associés.

2.3. Produits et publication multi-banque

Après la validation du partenariat, le concessionnaire accède à un espace centralisé lui permettant de gérer son catalogue. Il peut y créer, modifier et mettre à jour les informations relatives à ses produits, tout en assurant le suivi de leur stock.

Le catalogue des produits est géré indépendamment de leur diffusion auprès des banques. Un même produit peut ainsi être proposé à plusieurs banques sans qu'il soit nécessaire de créer plusieurs copies du produit. Le concessionnaire conserve donc une référence centrale de son catalogue tandis que chaque publication est associée au partenariat et à la marketplace concernée.

Cette organisation permet au concessionnaire de gérer le stock de chaque produit depuis une seule fiche centralisée. Il peut ainsi suivre la quantité totale, la quantité disponible à la vente et la quantité réservée, sans devoir mettre à jour séparément le même produit dans chaque marketplace bancaire. Lorsqu'une information du produit ou de son stock est modifiée, cette mise à jour est automatiquement prise en compte pour l'ensemble de ses publications autorisées.

Cependant, la publication d'un produit reste soumise à plusieurs conditions. Le concessionnaire doit être approuvé, le partenariat et le contrat doivent être valide, le store concerné doit être actif et la publication doit être approuvée par la banque.

Capture 5.4 — Création d'un produit et gestion du stock

Capture 5.5 — Validation bancaire d'une publication

2.4. Suivi des demandes de financement

Le concessionnaire dispose également une section dédié au suivi des demandes de financement associées à ses produits. Cette fonctionnalité constitue le principe de compte unique : les demandes provenant de différentes marketplaces partenaires peuvent être consultées depuis le même espace.

La section de suivi permet notamment de consulter les références des dossiers, les produits concernés, la banque associée, le client concerné, le montant demandé, la date de création et le statut du dossier.

Lorsqu'une demande associée à un produit concessionnaire est acceptée par la banque, le stock correspondant peut être réservé automatiquement. En revanche, une demande rejetée ne provoque aucune réservation.

Capture 5.6 — Suivi des demandes de financement par le concessionnaire

3. Parcours client et demande de financement

Le parcours client constitue un processus important de la plateforme. Il permet à un visiteur de découvrir les offres proposées dans la marketplace, de consulter et de comparer les produits disponibles, puis d'effectuer une simulation de financement. Lorsqu'il souhaite poursuivre sa démarche, il peut créer un compte client puis soumettre une demande de financement.

3.1. Consultation, comparaison et simulation

Depuis la marketplace publique, le visiteur peut parcourir les différents stores et consulter les produits proposés par les concessionnaires partenaires. Chaque fiche produit présente les informations utiles à la prise de décision, telles que les caractéristiques, le prix, les images et, selon le store, les conditions d'éligibilité associées.

Lorsque les modules correspondants sont activés, le visiteur peut sélectionner plusieurs produits afin de les comparer selon différents critères. Il peut également utiliser le simulateur de financement pour obtenir une estimation à partir du montant du produit, de la durée souhaitée et de l'apport personnel. Ces fonctionnalités restent accessibles avant la création d'un compte et permettent au visiteur d'étudier les offres avant d'entamer une demande de financement.

Capture 5.7 — Comparateur /simulateur

3.2. Inscription du client et activation du compte

Lorsqu'un visiteur souhaite déposer une demande de financement, il est orienté vers la création d'un compte client. Cette inscription s'effectue dans le contexte de la marketplace bancaire concernée afin de rattacher le compte au tenant correspondant.

Après validation du formulaire, un code de vérification à six chiffres est envoyé à l'adresse e-mail renseignée. Le compte est activé après la validation de ce code. Le client peut alors se connecter à son espace personnel et reprendre le processus de financement.

Capture 5.7 — Inscription et vérification du compte client

3.3. Constitution et soumission du dossier

Une fois authentifié, le client peut commencer sa demande depuis la fiche d'un produit ou à partir du résultat de sa simulation. Le dossier est créé initialement à l'état de brouillon, ce qui lui permet de compléter progressivement les informations requises et de déposer les documents justificatifs demandés.

Avant la soumission définitive, le système vérifie la présence des pièces obligatoires. Une fois le dossier complet, il est transmis à la banque pour traitement.

Figure 5.4 — Diagramme d'activité du parcours de demande de financement

La figure présente les principales étapes du parcours client, depuis la simulation jusqu'à la soumission du dossier et sa transmission à la banque.

3.4. Traitement de la demande par la banque

Après sa soumission, la demande est transmise à la banque concernée. L'Administrateur Banque peut alors consulter le détail du dossier ainsi que les documents fournis par le client avant de rendre sa décision. Lorsqu'elle est en attente de traitement, la demande peut être acceptée ou rejetée, avec l'ajout obligatoire d'un motif de rejet.

Figure 5.5 — Diagramme de séquence du traitement d'une demande de financement

Capture 5.8 — Simulation et création d'une demande de financement

Capture 5.9 — Téléversement des documents justificatifs

Capture 5.10 — Traitement et décision de la banque

4. Assistants conversationnels de la plateforme

La plateforme Matchia intègre deux solutions conversationnelles complémentaires répondant à des besoins distincts : un **assistant IA** destiné au pilotage du Back-office SaaS et un **chatbot public** accessible aux visiteurs des marketplaces bancaires. Bien que ces deux composants permettent une interaction en langage naturel, ils se distinguent par leurs objectifs, leurs utilisateurs cibles ainsi que par les mécanismes techniques sur lesquels ils reposent.

4.1. Assistant IA pour le pilotage du SaaS

L'assistant IA est intégré au Back-office SaaS et est destiné principalement à l'Administrateur SaaS. Il facilite l'accès aux informations de pilotage en permettant à l'administrateur de formuler directement ses questions en langage naturel. Ces interrogations peuvent notamment concerner les banques, les marketplaces, les demandes, les abonnements, les paiements, les stores ou encore les modules disponibles sur la plateforme.

Cet assistant s'appuie sur le modèle **Gemini**, appartenant à la famille des **LLM**. Grâce à ses capacités de traitement du langage naturel (**NLP**), le modèle analyse la question formulée par l'utilisateur, en détermine le sens et génère une requête permettant de récupérer les informations correspondantes dans la base de données. Les résultats obtenus sont ensuite reformulés afin de fournir une réponse compréhensible et directement exploitable par l'administrateur.

4.2. Fonctionnement et sécurisation de l'assistant IA

Lorsqu'une question est envoyée par l'administrateur, le backend commence par construire un schéma dynamique contenant uniquement les tables, les colonnes et les relations autorisées. Ce schéma, accompagné de la question formulée en langage naturel, est ensuite transmis au modèle Gemini.

À partir de ces informations, Gemini génère une proposition de requête SQL correspondant à la demande. Toutefois, cette requête n'est jamais exécutée directement sur la base de données. Elle est préalablement analysée par un validateur déterministe, chargé de vérifier qu'elle respecte les règles de sécurité définies par la plateforme.

Par ailleurs, les informations sensibles telles que les mots de passe, les tokens, les secrets, les clés, les données de vérification ainsi que certaines informations sont exclues du schéma transmis au modèle. Après validation, la requête est exécutée en lecture seule sur PostgreSQL, avec un contrôle du temps maximal d'exécution.

Les résultats obtenus sont ensuite transmis à Gemini afin de générer une réponse claire, précise et adaptée au contexte métier.

Figure 5.6 — Diagramme de séquence de l'assistant IA Text-to-SQL sécurisé

4.3. Chatbot public des marketplaces

Le chatbot public constitue le second composant conversationnel de la plateforme. Il est destiné aux visiteurs des marketplaces bancaires et apparaît uniquement lorsque le module Chatbot est activé pour le store consulté. Son rôle principal consiste à accompagner le visiteur dans la découverte des produits et à faciliter son accès aux différentes fonctionnalités disponibles au sein de la marketplace.

Ce chatbot public est un système conversationnel basé sur des règles métier. Son fonctionnement consiste à normaliser le message reçu, à rechercher des mots-clés et à identifier une intention prédéfinie afin de fournir une réponse adaptée.

Lorsqu'un visiteur envoie un message, le système identifie tout d'abord la marketplace et le store dans lesquels la conversation est effectuée. Le chatbot analyse ensuite le contenu du message afin d'en déterminer l'intention. Celle-ci peut notamment correspondre à la recherche d'un produit, à la consultation de sa disponibilité ou de ses caractéristiques, à l'identification du concessionnaire qui le propose, à une demande de comparaison ou encore à une orientation vers le simulateur de financement.

L'accès aux données reste limité au périmètre public de la marketplace consultée. Le chatbot refuse donc toute demande non autorisée.

Figure 5.7 — Fonctionnement du chatbot public dans une marketplace

5. Introduction

En conclusion, ce chapitre a permis de mettre en évidence l'ensemble des fonctionnalités métier développées pour les Concessionnaires et les Clients, ainsi que les mécanismes facilitant leurs interactions avec les banques. Il a également présenté l'apport de l'assistant IA et du chatbot dans l'amélioration de l'accès à l'information et de l'accompagnement des utilisateurs.

Chapitre 6 : DevOps, pipeline CI/CD et déploiement sur Microsoft Azure

1. Introduction

Ce chapitre présente la démarche DevOps adoptée pour automatiser le processus de livraison et de déploiement de Matchia. Il décrit la conteneurisation de l'application avec Docker, la mise en place du pipeline CI/CD avec Jenkins, les tests et l'analyse de qualité avec SonarQube, ainsi que la publication des images Docker et le déploiement de la plateforme sur Microsoft Azure.

2. Démarche DevOps et principes de la CI/CD

Le DevOps est une approche visant à rapprocher les activités de développement (*Development*) et d'exploitation (*Operations*) afin d'améliorer la collaboration et d'automatiser les différentes étapes du cycle de livraison d'une application. Elle permet notamment d'enchaîner de manière cohérente la construction, les tests, le contrôle de la qualité, la conteneurisation et le déploiement, dans le but d'améliorer la rapidité, la fiabilité et la reproductibilité des mises en production.

Dans le cadre de Matchia, cette démarche repose principalement sur GitHub, Jenkins, SonarQube, Docker, Docker Hub et Microsoft Azure. L'objectif est de mettre en place un processus de livraison automatisé, fiable et reproductible, tout en réduisant les interventions manuelles lors du déploiement des nouvelles versions.

2.1. Intégration continue

L'intégration continue consiste à intégrer régulièrement les modifications du code source et à les soumettre automatiquement à un ensemble de vérifications. Elle permet de détecter rapidement les erreurs introduites au cours du développement et de vérifier que la nouvelle version peut être construite correctement.

Dans Matchia, le processus d'intégration continue est orchestré par Jenkins et déclenché à la suite d'un *push* effectué sur le dépôt GitHub. Il comprend notamment la récupération du code

source, la construction de l'application, l'exécution des tests backend et l'analyse de la qualité du backend et du frontend avec SonarQube.

2.2. Livraison et déploiement continu

Le déploiement continu intervient après l'intégration continue et vise à automatiser les étapes nécessaires à la mise à disposition d'une version validée de l'application. Dans le cadre de Matchia, cette phase comprend la création des images Docker du frontend et du backend, leur publication sur Docker Hub, puis le déploiement des nouvelles versions dans l'environnement Microsoft Azure. Cette automatisation permet de fiabiliser les mises à jour et de réduire les interventions manuelles lors du déploiement.

3. Conteneurisation avec Docker

3.1. Principe de la conteneurisation

La conteneurisation est une technique qui consiste à regrouper une application ainsi que l'ensemble des éléments nécessaires à son fonctionnement dans une unité isolée appelée conteneur. Contrairement à une machine virtuelle, un conteneur ne nécessite pas un système d'exploitation complet. Il partage le noyau du système hôte tout en disposant de son propre environnement d'exécution, ce qui le rend plus léger, plus rapide à démarrer et plus simple à déployer.

Docker est la technologie utilisée dans Matchia pour mettre en œuvre cette conteneurisation. Elle permet de construire un environnement d'exécution reproductible et indépendant de la machine sur laquelle l'application est exécutée.

3.2. Dockerfile, image et conteneur

La création d'un conteneur Docker repose principalement sur trois éléments : le Dockerfile, l'image Docker et le conteneur.

Le Dockerfile est un fichier contenant les différentes instructions nécessaires à la préparation de l'environnement d'exécution de l'application. Il précise notamment l'image de base utilisée, les dépendances à installer, les fichiers à copier, les commandes de compilation et la commande permettant de démarrer l'application.

À partir du Dockerfile, Docker construit une image Docker. Une image est un modèle immuable contenant l'application ainsi que les dépendances nécessaires à son fonctionnement.

Lorsqu'une image est démarrée, Docker crée un conteneur. Le conteneur représente donc l'instance en cours d'exécution d'une image Docker.

3.3. Conteneurisation du frontend et du backend

Dans Matchia, le frontend et le backend sont conteneurisés séparément et disposent chacun de leur propre Dockerfile. Cette séparation permet de construire, versionner et déployer les deux composants indépendamment.

Le backend, développé avec Spring Boot et Java 17, est compilé puis exécuté dans un environnement Java adapté. Le frontend, développé avec React et TypeScript, est quant à lui compilé afin de générer les fichiers statiques de production, qui sont ensuite servis à l'aide du serveur web Nginx.

Cette organisation facilite la maintenance de la plateforme, puisqu'une modification du frontend peut être déployée indépendamment d'une modification du backend, et inversement.

3.4. Orchestration avec Docker Compose

Pour simplifier l'exécution de plusieurs conteneurs en environnement local, Matchia utilise Docker Compose. Celui-ci permet de regrouper dans un même fichier la configuration des différents services de la plateforme.

Le fichier Docker Compose définit notamment les conteneurs du frontend et du backend, leurs ports, les variables d'environnement et le réseau permettant leur communication. Une seule commande permet ainsi de construire et de démarrer l'ensemble des services nécessaires au fonctionnement local de la plateforme.

4. Mise en place du pipeline CI/CD avec Jenkins

Jenkins constitue l'outil central d'orchestration de la chaîne CI/CD. Il est chargé d'exécuter automatiquement les différentes opérations nécessaires à la validation et à la livraison d'une nouvelle version de l'application. Les différentes étapes à exécuter sont décrites dans un

Jenkinsfile intégré au dépôt du projet, ce qui permet de versionner la définition du pipeline avec le code source.

4.1. Déclenchement automatique avec GitHub Webhook

Le déclenchement du pipeline est automatisé à l'aide d'un Webhook GitHub. À chaque *push* effectué sur le dépôt du projet, GitHub envoie automatiquement une notification à Jenkins afin de signaler la disponibilité d'une nouvelle version du code source.

À la réception de cet événement, Jenkins lance le pipeline défini dans le Jenkinsfile et récupère la version la plus récente du projet. Les différentes étapes de construction, de test, d'analyse de qualité, de conteneurisation et de déploiement sont ensuite exécutées automatiquement selon l'ordre défini dans le pipeline.

Ce mécanisme évite le déclenchement manuel du pipeline et garantit que chaque nouvelle version déposée sur GitHub passe par le même processus automatisé de validation, de construction et de livraison.

4.2. Construction du backend et du frontend

Après la récupération du code source, Jenkins procède à la construction des deux composants. Le backend Spring Boot est construit avec Maven, tandis que le frontend est construit avec Node.js.

Lors de la construction du frontend destiné à l'environnement de production, l'adresse du backend déployé sur Azure est intégrée à sa configuration.

4.3. Exécution des tests backend

Une fois la phase de construction réalisée, le pipeline exécute automatiquement les tests du backend avec Maven. Cette étape permet de vérifier le comportement de l'application avant de poursuivre la chaîne de livraison.

La couverture des tests backend est également générée avec JaCoCo. Les informations obtenues peuvent ensuite être exploitées par SonarQube afin de compléter l'évaluation de la qualité du code.

5. Analyse de la qualité avec SonarQube

Afin de compléter les étapes de compilation et de test, SonarQube est intégré à la chaîne CI/CD pour analyser la qualité du code source. Deux analyses distinctes sont réalisées : une pour le backend et une pour le frontend.

SonarQube permet notamment d'identifier les problèmes susceptibles d'affecter la fiabilité, la sécurité et la maintenabilité du projet, tels que les bugs potentiels, les vulnérabilités et les duplications de code. Les résultats obtenus fournissent ainsi des indicateurs permettant de suivre et d'améliorer progressivement la qualité du code.

6. Construction et publication des images Docker

Après les étapes de construction, de test et d'analyse de qualité, Jenkins procède à la création des images Docker du frontend et du backend. Chaque composant dispose ainsi de sa propre image contenant l'environnement nécessaire à son exécution.

Une fois construites, les images Docker du frontend et du backend sont publiées sur Docker Hub, utilisé comme registre d'images pour la plateforme Matchia. Jenkins s'authentifie auprès du registre puis publie automatiquement les images correspondant aux deux composants de l'application.

Les images sont associées à des tags permettant d'identifier les différentes versions produites par le pipeline. Elles peuvent ensuite être récupérées depuis Docker Hub afin d'être utilisées lors du déploiement des services sur Microsoft Azure.

7. Déploiement sur Microsoft Azure

7.1. Choix de la plateforme Cloud

Après la mise en place de la chaîne CI/CD, le déploiement de Matchia nécessite un environnement capable d'héberger les différents services de la plateforme et de prendre en charge leur évolution. Le recours à une infrastructure Cloud permet notamment de disposer de ressources accessibles à distance, configurables et adaptées au déploiement d'applications conteneurisées.

Dans le cadre de Matchia, Microsoft Azure a été retenu comme plateforme Cloud en raison de sa prise en charge des applications conteneurisées et de son intégration avec les outils utilisés dans la chaîne de déploiement. Il permet notamment d'héberger séparément le frontend et le

backend, de configurer leurs paramètres d'exécution et de les mettre à jour à partir des images Docker produites par le pipeline.

7.2. Déploiement avec Azure Container Apps

Pour le déploiement de la couche applicative, Matchia utilise Azure Container Apps, un service Microsoft Azure destiné à l'exécution d'applications conteneurisées.

L'architecture repose sur deux Container Apps distinctes. La première exécute l'image Docker du frontend, dans laquelle les fichiers de production de l'application React sont servis par Nginx. La seconde exécute l'image Docker du backend, qui expose les services REST nécessaires au fonctionnement de la plateforme.

Cette séparation permet de gérer et de mettre à jour indépendamment le frontend et le backend. Une fois déployé, le frontend communique avec le backend à travers l'URL exposée par le service backend sur Azure.

7.3. Base de données PostgreSQL sur Azure

La persistance des données de Matchia repose sur une base de données PostgreSQL hébergée dans l'environnement Azure. Celle-ci est séparée des conteneurs applicatifs afin de conserver un cycle de vie indépendant de celui du frontend et du backend.

7.4. Automatisation du déploiement

Après la publication des images Docker, Jenkins poursuit automatiquement le processus de livraison vers Microsoft Azure. À l'aide d'Azure CLI, le pipeline met à jour les Container Apps du frontend et du backend avec les nouvelles images produites.

Chaque déploiement entraîne la création d'une nouvelle révision du service concerné sur Azure Container Apps. Cette étape permet d'automatiser la mise à disposition d'une nouvelle version de Matchia et complète ainsi la chaîne allant du *push* du code source jusqu'au déploiement dans l'environnement Cloud.

8. Architecture complète de déploiement

9. Conclusion

En conclusion, ce chapitre nous a permis de présenter la démarche DevOps adoptée pour la plateforme Matchia, en partant de la conteneurisation de l'application avec Docker jusqu'à son déploiement sur Microsoft Azure. Nous avons également mis en place un pipeline **CI/CD** avec Jenkins permettant d'automatiser les différentes étapes de construction, de test, d'analyse de la qualité avec SonarQube, de création et de publication des images Docker, puis de déploiement.